G09/G16 MATLAB Toolbox

A parsing and visualisation toolbox for Gaussian 09/16 output files

Sebastiano Trusso
CNR – Istituto per i Processi Chimico-Fisici (IPCF), Messina, Italy

Version 1.1 — 2026

Contents

1 Introduction **4**

2 Which `.in` keywords generate which `.out` section **5**

3 File I/O utilities **7**
3.1 G09_read_lines . . . 7
3.2 G09_check_gaussian_match / G16_check_gaussian_match . . . 7
3.3 split_link1_log (shared, no G09_/G16_ prefix) . . . 7

4 Core data extraction **9**
4.1 G09_route / G16_route . . . 9
4.2 G09_structure / G16_structure . . . 9
4.3 G09_energy / G16_energy . . . 10
4.4 G09_charge_mult / G16_charge_mult . . . 11
4.5 G09_convergence / G16_convergence . . . 11
4.6 G09_dipole_polar / G16_dipole_polar . . . 12
4.7 G09_nmodes / G16_nmodes . . . 12
4.8 G09_spectra / G16_spectra . . . 14
4.9 G_spectra_nm (shared, no G09_/G16_ prefix) . . . 14

5 Atomic charges **16**
5.1 G09_charges / G16_charges . . . 16
5.2 G09_charges_fchk / G16_charges_fchk . . . 17

6 Formatted checkpoint (.fchk) support **19**
6.1 G09_fchk_read / G16_fchk_read . . . 19

7 Molecule and normal-mode visualisation **21**
7.1 G09_draw_molecule / G16_draw_molecule . . . 21
7.2 G09_draw_mode / G16_draw_mode . . . 22
7.3 G09_modeViewer / G16_modeViewer . . . 23
7.4 G09_animate_mode / G16_animate_mode . . . 23
7.5 G09_draw_deformation / G16_draw_deformation . . . 25

8 au2SI: atomic units ↔ SI conversions **27**
8.1 AU_convert . . . 27
8.2 AU_table . . . 29

8.3 G_gaussian_field_convert . . . 30
8.4 Integration with the G09/G16 Toolbox . . . 31
8.5 CODATA 2022 fundamental constants . . . 32
8.6 Python port (g16_au_convert / g16_au_table / g16_gaussian_field_convert) . . . 33

9 MOSurface: real-space orbital, density, and ESP isosurfaces 34
9.1 G_draw_mo_surface . . . 34
9.2 G_draw_density_surface . . . 37
9.3 G_draw_esp_surface . . . 39
9.4 G_draw_cube_surface . . . 42
9.5 G_relight . . . 44
9.6 Python port (MOSurface/python/) . . . 45

10 RamanBrowser: interactive Raman/IR stick-spectrum browser 47
10.1 G_raman_browser . . . 47
10.2 Python port (RamanBrowser/python/) . . . 48

11 Orbital energy analysis 50
11.1 G09_orbital_energies / G16_orbital_energies . . . 50
11.2 G09_draw_orbital / G16_draw_orbital . . . 51

12 Response properties (G16 only) 52
12.1 G16_hyperpolar . . . 52
12.2 G16_tddft . . . 52
12.3 G16_td_plot . . . 53
12.4 G16_td_levels . . . 54

13 Structural analysis utilities 56
13.1 G09_get_bond_length / G16_get_bond_length . . . 56
13.2 G09_get_angle / G16_get_angle . . . 56
13.3 G09_get_dihedral / G16_get_dihedral . . . 57
13.4 G09_nbo_bonds / G16_nbo_bonds . . . 57
13.5 G09_gaussian_version / G16_gaussian_version . . . 58
13.6 G09_restart / G16_restart . . . 58
13.7 G09_available_data / G16_available_data . . . 59
13.8 G_read_input (shared, no G09_/G16_ prefix) . . . 60
13.9 G09_list / G16_list . . . 61

14 One-call data extraction 62
14.1 G09_read_all / G16_read_all . . . 62
14.2 G09_batch_read_all / G16_batch_read_all . . . 63

15 Report generation 64
15.1 G09_write_report / G16_write_report . . . 64
15.2 Function reference . . . 65
15.3 Selected signatures . . . 67
15.4 Notes on the port . . . 68

16 Worked examples 72
16.1 example_1.m — Structure, vibrational mode, and charges . . . 72
16.2 example_2.m — Infrared and Raman spectra . . . 73
16.3 example_3.m — Energetics . . . 74
16.4 example_4.m — Recovering and restyling graphics handles . . . 76

17 References **78**

18 Publication and licensing **79**
18.1 Declaration of Generative AI and AI-assisted technologies in the writing process 79
18.2 Note on the Use of AI-Assisted Development Tools 79
18.3 Scope and disclaimer 79
18.4 Licence 79
18.5 Citation 79

1 Introduction

This manual documents the **G09/G16 Toolbox**, a set of MATLAB functions for parsing, analysing, and visualising Gaussian 09 and Gaussian 16 output files (`.out`/`.log`) and formatted checkpoint files (`.fchk`). The toolbox covers:

- extraction of molecular geometry, energies, thermochemistry, dipole moment, polarisability, and hyperpolarisability;
- vibrational analysis: normal modes, IR/Raman spectra, and interactive mode visualisation;
- molecular-orbital energy diagrams and TD-DFT excited-state analysis;
- 3D molecular structure rendering (CPK ball-and-stick models) with atomic charge and dipole overlays;
- utility functions for bond-length tables, Gaussian version detection, restart-file generation, and toolbox self-documentation;
- real-space molecular-orbital, electron-density, and electrostatic- potential isosurfaces evaluated directly from `.fchk` basis-set data (`MOSurface`, §9);
- an interactive click-to-select Raman/IR stick-spectrum browser (`RamanBrowser`, §10);
- atomic-unit ↔ SI conversions for the physical quantities computational chemistry results are commonly reported in (`au2SI`, §8).

Function names follow the `G09_xxx`/`G16_xxx` convention, indicating which Gaussian version's output format the function targets. Where the two versions behave identically, this manual documents them in a single combined subsection (`G09_xxx / G16_xxx`); where behaviour differs, the differences are called out explicitly.

Every parsing function that reads a `.out`/`.log` file accepts an optional `'Lines'` Name-Value parameter: a cell array of pre-read file lines. This lets `G09_read_all`/`G16_read_all` read the file from disk once and reuse the same lines across every sub-parser, instead of each function re-reading and re-splitting the file independently. Passing `'Lines'` is entirely optional — omitting it falls back to reading the file normally.

Wrong-toolbox warning. Because Gaussian 09 and Gaussian 16 output formats differ, using the wrong toolbox on a file (e.g. `G16_energy` on a Gaussian 09 file) can silently misparse it rather than raising an obvious error — for example, `G16_dipole_polar` on a Gaussian 09 file returns `NaN` for the polarisability instead of an error. To catch this, every function that reads a file from disk checks the Gaussian-version citation line (`"Gaussian NN, Revision X.YY,"`) it just read and prints a non-blocking `warning(...)` if `NN` does not match the toolbox being used, suggesting the other one instead. The check runs only once per actual disk read (see Section 3.2), so `G09_read_all`/`G16_read_all` produce a single warning, not one per sub-parser.

2 Which `.in` keywords generate which `.out` section

Each extraction function reads one specific section of the `.out`/ `.log` file, identified by a fixed header string or line pattern. Most of those sections only appear if the corresponding Gaussian route keyword was present in the `.in`/`.gjf` job that produced the file — otherwise the function raises an error (or, for optional response-property fields, returns `NaN`/empty rather than failing). The table below maps each function to the `.out` section it reads and the `.in` keyword(s) that generate it. Function names omit the `G09_`/`G16_` prefix (identical for both versions unless noted).

Function	`.out` section read	`.in` keyword	Notes
`structure`	`"Standard orientation"` / `"Input orientation"`	(default)	Always printed. `nosymm`/`nosym` suppresses "Standard orientation", leaving only "Input orientation"
`energy` (SCF)	`"SCF Done:"`	(default)	Printed by any HF/DFT energy evaluation (single point, opt, freq, ...)
`energy` (thermochemistry)	`"Zero-point correction="`, `"Sum of electronic and ... Energies="`	`freq`	ZPE, H, G, S
`charges` (Mulliken)	`"Mulliken charges:"`	(default)	
`charges` (APT)	`"APT charges:"`	`freq`	Printed automatically with any frequency job (dipole-derivative charges) — no separate `pop=apt` needed
`dipole_polar` (dipole)	`"Dipole moment (...)"`	(default)	
`dipole_polar` (static Alpha)	`"Exact polarizability:"`	`polar`	
`dipole_polar` (dynamic Alpha)	`"Alpha(-w,w) frequency N"`	`polar` `cphf=rdfreq`	Requires an explicit frequency list (a.u.) after the route/title/charge-multiplicity block in the `.in` file
`nmodes`	`"and normal coordinates:"`	`freq`	
`spectra` (IR)	`"Harmonic frequencies ... IR intensities"`	`freq`	
`spectra` (Raman)	`"Harmonic frequencies ... Raman scattering"`	`freq=raman`	
`orbital_energies`	`"Alpha occ./virt. eigenvalues"` (+ `Beta ...` for unrestricted)	(default)	Full eigenvalue list is printed unconditionally, not gated by `pop=full`
`convergence`	`"Item Value Threshold Converged?"`	`opt`	

Function	.out section read	.in keyword	Notes
charge_mult	charge/multiplicity echo near the top of the file	(default)	Mirrors the .in charge/-multiplicity line
route	route-section echo near the top of the file	(default)	
get_bond_length	derived from the structure section	(default)	Purely geometric (covalent radii); no dedicated Gaussian section
nbo_bonds	"Natural Bond Orbitals (Summary)"	pop=nbo	See Section 13.4
gaussian_version	"Gaussian NN, Revision X.YY," citation line	(default)	
hyperpolar (vibrational Beta, G16 only)	"Diagonal vibrational hyperpolarizability:"	polar freq	Present even without cphf=rdfreq
hyperpolar (electronic Beta, G16 only)	"First dipole hyperpolarizability, Beta ...", "Beta(0;0,0)", "Beta(-w;w,0)"	polar field=... cphf=rdfreq	Needs the field/frequency list in the .in file; polar alone gives only the vibrational Beta above
tddft (G16 only)	"Excited State N: ..."	td (e.g. td=(nstates=N))	
restart	reuses the structure/route/charge_mult sections above	(any completed step)	

.fchk functions. fchk_read and charges_fchk (provided in both G09/ and G16/) do not read .out/.log sections at all — they read a formatted checkpoint file, produced by running formchk on the binary .chk file written by the job's %chk=... link0 command. No specific route keyword is required beyond %chk=... itself.

Evidence, not assumption. The keyword requirements above were verified directly against real Gaussian output files rather than inferred from general documentation — e.g. APT charges and the full orbital-eigenvalue list turned out to be printed unconditionally (no pop=apt/pop=full needed), and the vibrational vs. electronic hyperpolarisability distinction was confirmed by comparing a file run with plain polar against one run with field=... cphf=rdfreq, only the latter containing the full electronic Beta tensor.

3 File I/O utilities

3.1 G09_read_lines

```
lines = G09_read_lines(filename)
```

Low-level reader used internally by every other `G09_XXX` parser. Reads a Gaussian 09 output file and returns a cell array of lines, with line endings stripped. Transparently handles CRLF line endings (as produced by Windows G09W builds) and ISO-8859-1 (Latin-1) encoding.

> G16 parsers read files directly with the platform default encoding rather than calling a separate `G16_read_lines` helper; the `'Lines'` parameter interface is identical across both toolboxes.

3.2 G09_check_gaussian_match / G16_check_gaussian_match

```
ok = G09_check_gaussian_match(lines, filename)
ok = G16_check_gaussian_match(lines, filename)
```

Internal helper, called automatically by every function that reads a `.out`/`.log` file from disk (not part of the typical user-facing API, but documented here for completeness). Scans the already-read `lines` (no extra disk I/O) for the `"Gaussian NN, Revision X.YY,"` citation line printed near the top of every output file. If found and `NN` does not match the toolbox (9 for `G09_*`, 16 for `G16_*`), prints a non-blocking warning suggesting the other toolbox; otherwise does nothing. Returns `true` if no mismatch was detected (or the version is unknown), `false` if a warning was issued.

> Because this check lives inside each function's own "read the file fresh from disk" branch, it only fires when a file is actually opened — if a function receives pre-read `lines` via the `'Lines'` parameter (e.g. every sub-parser called by `G09_read_all`/`G16_read_all`), it skips both the disk read and the check, so a single call to `read_all` produces exactly one warning, not one per sub-parser.

3.3 split_link1_log (shared, no G09_/G16_ prefix)

```
files = split_link1_log(logfile)                    % automatic naming
files = split_link1_log(logfile, 'naming', 'chk') % names from %chk
files = split_link1_log(logfile, 'naming', 'field', ...
                        'prefix', 'NMA_dip')      % names from field
```

Splits a Gaussian output that contains several jobs chained with `--Link1--` into one log file per job, so that every job can be read by the single-file parsers of the toolbox. Jobs are separated at the `Entering Link 1` lines; the file header is kept with the first job. The internal Link1 step that Gaussian creates for the Freq part of an `Opt Freq` route carries the same title and `%chk` (or none) and is kept together with its optimisation. Each output file is named after a key read from its job:

Option	Default	Description
'naming'	'auto'	'chk': name of the job's .chk file (e.g. NMA_TD_trans.log); 'field': <prefix>_Fp020.log, <prefix>_Fm040.log, ... from the field value in the job title; 'auto': 'field' if the title pattern is found in the first job, otherwise 'chk'
'pattern'	'scan along X: N = (-?\d+)'	Regular expression with one token that captures the field value N in the title ('field' naming)
'prefix'	log name without _scan_all	Prefix of the output names ('field' naming)
'outdir'	folder of the input	Output folder (created if missing)

Returns a cell array with the full names of the logs written and prints one line per job with the number of `Normal termination` lines and a status (`OK`, `ERROR termination`, `n of m steps terminated`, `incomplete / still running`, plus `stationary point` for converged optimisations).

```
files = split_link1_log('NMA_WB97XD_DEF2TZVP_TD_ALL.LOG', 'naming', '
    chk');
T = G16_td_plot(files{2});          % read the second job on its own
```

> Gaussian wraps Link0 lines at 80 columns, so a long `%chk` path is printed on two or more lines; the function joins the continuation lines until the name ends in `.chk`. Paths in the `%chk` line may use either \ or `/`. A job without a `%chk` line inherits the key of the previous one; if two jobs share a key, _2, _3, ... are appended, and the input log is never overwritten. The function uses an `arguments` block and needs MATLAB R2019b or later. Most parsers of the toolbox can also read a single job of a chain directly through their 'job' option (`G16_scan_read`, `G16_irc_read`, `G16_td_plot`, `G16_td_levels`); splitting is convenient when the same job is read many times or by other programs.

4 Core data extraction

4.1 G09_route / G16_route

```
route = G09_route(filename)
route = G16_route(filename)
route = G16_route(filename, 'Lines', lines)
```

Extracts the complete route section (the # line(s) between the two ---- separators) as a single string.

Option	Default	Description
'Lines'	{}	Pre-read file lines (Section 3.1); read normally if omitted

```
r = G09_route('indaco.log');
% '# opt freq=raman b3lyp/6-311++g(d,p) nosymm geom=connectivity field
   =x+50'
```

> Both G09_route/G16_route and G09_charge_mult/ G16_charge_mult now accept the 'Lines' option, consistent with the rest of the toolbox.

4.2 G09_structure / G16_structure

```
mol = G09_structure(filename)
mol = G09_structure(filename, 'step', 'last')
mol = G09_structure(filename, 'step', 'first')
mol = G09_structure(filename, 'step', N)

mol = G16_structure(filename)
mol = G16_structure(filename, 'orientation', 'standard')
mol = G16_structure(filename, 'orientation', 'input')
```

Extracts the molecular geometry from a Gaussian output file.

> **Difference:** Gaussian 09 writes only "Input orientation:" blocks (never "Standard orientation:"), so G09_structure has no 'orientation' option. Gaussian 16 can write either, so G16_structure exposes 'orientation' = 'standard' (default, falls back to input if absent) | 'input' | 'auto'.

Option	Default	Description
'step'	'last'	Which geometry block to extract: 'first', 'last', or an integer index
'orientation'	'auto' (G16 only)	'standard' | 'input' | 'auto'
'Lines'	{}	Pre-read file lines; read normally if omitted

Field	Type	Description
`.symbols`	cell	Atomic symbols
`.xyz`	double	Coordinates in Å (X Y Z)
`.Z`	int	Atomic numbers
`.Natoms`	int	Number of atoms
`.step`	int	Index of the extracted step (1-based)
`.n_steps`	int	Total geometry blocks found in the file
`.orientation`	char	`'Input orientation'` (always, G09) or `'standard'/'input'` (G16)
`.filename`	char	Source file name

```
mol = G09_structure('indaco.log');
mol = G16_structure('calc.out', 'orientation', 'input');
```

4.3 G09_energy / G16_energy

```
en = G09_energy(filename)
en = G09_energy(filename, 'step', 'last')
en = G09_energy(filename, 'step', N)
```

Extracts SCF energy and thermochemistry data.

Field	Type	Description
`.SCF`	double	SCF energy (Hartree)
`.method`	char	Method string, e.g. `'RB3LYP'`
`.ZPE_corr`	double	Zero-point correction (Hartree)
`.U_corr`	double	Thermal correction to energy (Hartree)
`.H_corr`	double	Thermal correction to enthalpy (Hartree)
`.G_corr`	double	Thermal correction to Gibbs free energy (Hartree)
`.E0`	double	SCF + ZPE (Hartree)
`.U`	double	SCF + U_corr (Hartree, G16 only)
`.H`	double	SCF + H_corr (Hartree)
`.G`	double	SCF + G_corr (Hartree)
`.S_JmolK`	double	Entropy S ($\mathrm{J\,mol^{-1}K^{-1}}$, G09)
`.ZPE_kJ`	double	Zero-point energy (kJ/mol)
`.T`, `.P`	double	Temperature (K), pressure (atm)
`.has_thermo`	logical	`false` for opt-only jobs (no freq); `*_corr`, `E0`, `U`, `H`, `G`, `T`, `P` are `NaN` in that case
`.G_kJ`, `.H_kJ`	double	G, H in kJ/mol (G09)
`.filename`	char	

Option	Default	Description
'step'	'last'	Which SCF block to extract: 'last', 'first', or integer N
'Lines'	{}	Pre-read file lines

4.4 G09_charge_mult / G16_charge_mult

```
[charge, mult] = G09_charge_mult(filename)
[charge, mult] = G16_charge_mult(filename)
[charge, mult] = G16_charge_mult(filename, 'Lines', lines)
```

Extracts molecular charge and spin multiplicity from the "Charge = ... Multiplicity = ..." line. Format identical across G09 and G16.

```
[q, m] = G09_charge_mult('indaco.log');   % q = 0, m = 1
```

4.5 G09_convergence / G16_convergence

```
cv = G09_convergence(filename)
cv = G09_convergence(filename, 'plot', true)

cv = G16_convergence(filename)
cv = G16_convergence(filename, 'plot', true)
```

Extracts geometry-optimisation convergence criteria (Max/RMS Force, Max/RMS Displacement) at every optimisation step, together with their thresholds and whether/when the job converged. Block format identical across G09 and G16.

Option	Default	Description
'plot'	false	Plot the four convergence series against their thresholds

Field	Type	Description
.MaxForce, .RMSForce	vector	Max / RMS force per step (a.u.)
.MaxDisp, .RMSDisp	vector	Max / RMS displacement per step (a.u.)
.thr_MaxForce, .thr_RMSForce, .thr_MaxDisp, .thr_RMSDisp	double	Corresponding convergence thresholds
.converged	logical	Whether all four criteria were satisfied
.conv_step	int/NaN	Step at which convergence was reached
.Nsteps	int	Number of optimisation steps found
.filename	char	

```
cv = G16_convergence('V_E00t.out', 'plot', true);
cv.converged
cv.conv_step
```

4.6 G09_dipole_polar / G16_dipole_polar

```
dp = G09_dipole_polar(filename)
dp = G09_dipole_polar(filename, 'units', 'Debye')

dp = G16_dipole_polar(filename)
dp = G16_dipole_polar(filename, 'units', 'SI')
```

Extracts the dipole moment and polarisability tensor.

> **Differences between G09 and G16:** G09 prints `"Exact polarizability:"` as six values on a single line (upper triangle, row order), rather than G16's formatted `Alpha(0;0)/Alpha(-w;w)` blocks. G09 has no dynamic (laser-frequency-dependent) polarisability unless requested via `Polar=OptRot` or similar keywords, and additionally prints a `"Diagonal vibrational polarizability:"` line (3 values, returned in `.alpha_vib`). An approximate CPHF tensor, if printed by G09 (`"Approx polarizability:"`), is returned in `.alpha_approx`. G16 instead reports full dynamic polarisability entries per laser frequency (`.alpha_dyn`) and modal derivatives `.dmu_dQ`/`.dalpha_dQ` (related to IR intensities and Raman activities; for exact scaled values use `G16_spectra` instead).

Option	Default	Description
`'units'`	`'au'`	`'au'` | `'Debye'` (dipole only, G09) / `'Debye'` (full, G16) | `'SI'`
`'Lines'`	{}	Pre-read file lines

Field	Description
`.mu_x`, `.mu_y`, `.mu_z`, `.mu_tot`, `.mu_units`	Dipole components, magnitude, unit string
`.alpha_iso`, `.alpha_aniso`	Isotropic polarisability, anisotropy
`.alpha_tensor`, `.alpha_units`	3×3 static tensor (a.u.), unit string
`.alpha_vib`, `.has_alpha_vib`	Diagonal vibrational polarisability (a.u.), logical (G09)
`.alpha_approx`	CPHF approximate tensor (a.u.), `NaN` if absent (G09)
`.alpha_dyn`, `.N_dyn`	Struct array of dynamic polarisability entries (per laser frequency), count (G16)
`.dmu_dQ`, `.dalpha_dQ`, `.has_deriv`	Dipole/polarisability derivatives w.r.t. normal modes, logical (G16)
`.filename`	

4.7 G09_nmodes / G16_nmodes

```
nm = G09_nmodes(filename)
nm = G09_nmodes(filename, 'modes', [5 10 20])

nm = G16_nmodes(filename)
```

```
nm = G16_nmodes(filename, 'section', 'last')
```

Extracts normal-mode displacement vectors.

> G09 writes only one *Harmonic frequencies* section per file (even for opt+freq jobs), so `G09_nmodes` has no `'section'` option — it is implicit. G16 may contain more than one such section (e.g. a preliminary and a final one) and defaults to the last.

Option	Default	Description
`'modes'`	`[]`	Restrict output to specific mode indices
`'section'`	`'last'` (G16 only)	`'last'` | `'first'`
`'Lines'`	{}	Pre-read file lines

Field	Description
`.freq`, `.IR`, `.Raman`	Frequencies (cm^{-1}), IR intensities (KM/Mole), Raman activities ($Å^4$/AMU, `[]` if absent)
`.redmass`, `.frcconst`	Reduced masses (AMU), force constants (mDyne/Å)
`.symmetry`	Symmetry labels
`.disp`	$[N_{\text{atoms}} \times 3 \times N_{\text{modes}}]$ Cartesian displacement vectors
`.Nmodes`, `.Natoms`, `.has_Raman`, `.filename`	
`.RamanFr`	$[N_{\text{modes}} \times K]$ per-incident-light-frequency Raman activities ($Å^4$/AMU), `[]` if the file has no `CPHF=RdFreq` data. Column 1 is always the static (0 cm^{-1}) value — numerically identical to `.Raman` — columns 2..K are the dynamic/pre-resonance values, one per incident wavelength actually requested
`.IncidentLight`	$[1 \times K]$ incident light frequencies in cm^{-1}, as printed on Gaussian's own *"Incident light (cm**-1): ..."* line (`[]` if absent); `IncidentLight(1)` is always `0.00`
`.has_RamanFr`	Logical, true if `.RamanFr` is non-empty

> **Static vs. dynamic (CPHF=RdFreq) Raman activities.** With `Freq=Raman` plus `CPHF=RdFreq` (e.g. a `"785 nm"` line in the input stream), Gaussian's own output prints, for every mode, both a generic `Raman Activ --` row (numerically the STATIC, 0 cm^{-1} value) and explicit `RamAct Fr= 1--`/`RamAct Fr= 2--` rows — Fr=1 repeating the same static value, Fr=2 being the genuinely frequency-dependent one at the requested wavelength. Earlier versions of `G09_nmodes`/`G16_nmodes` read only the generic row (per their own inline comment, *"RamAct Fr=... lines are skipped"*), so `.Raman` was always the static value even for a `CPHF=RdFreq` job, with the dynamic value silently unavailable. Verified directly on a 785 nm job (`MPBA_SH.out`): mode 1 has `.Raman` = 1.7384 (static) but `.RamanFr(1,2)` = 1.8922 (785 nm, from the same file) — a real, non-negligible difference. `.IR` has no such

ambiguity: Gaussian prints only one `IR Inten --` row (IR intensity comes from the dipole derivative, not the frequency-dependent polarizability, so it is not itself computed at more than one incident frequency).

4.8 G09_spectra / G16_spectra

```
sp = G09_spectra(filename)
sp = G09_spectra(filename, Name, Value, ...)

sp = G16_spectra(filename)
sp = G16_spectra(filename, 'section', 'last')
```

Extracts IR/Raman line spectra and generates Lorentzian-broadened continuous spectra.

G09 writes only one Harmonic-frequencies section per file, so `G09_spectra` needs no `'section'` parameter; `G16_spectra` exposes it to select which section to use when more than one is present.

Option	Default	Description
`'FWHM'`	`10`	Lorentzian FWHM (cm^{-1})
`'xmin'`	`0`	Lower wavenumber limit (cm^{-1})
`'xmax'`	`4000`	Upper wavenumber limit (cm^{-1})
`'dx'`	`1`	Grid step (cm^{-1})
`'normalize'`	`false`	Normalise continua to maximum = 1
`'plot'`	`false`	Generate figure
`'section'`	`'last'` (G16 only)	`'last'` | `'first'`
`'Lines'`	{}	Pre-read file lines

Field	Description
`.freq`, `.IR`, `.Raman`	Frequencies (cm^{-1}), IR intensities, Raman activities (`[]` if absent)
`.has_Raman`, `.Nmodes`	Logical, number of modes
`.x`, `.IR_cont`, `.Raman_cont`	Wavenumber grid, broadened IR/Raman continua
`.FWHM`, `.filename`	

```
sp = G16_spectra('calc.out', 'FWHM', 15, 'xmin', 400, 'plot', true);
```

4.9 G_spectra_nm (shared, no G09_/G16_ prefix)

```
sp = G_spectra_nm(nm)
sp = G_spectra_nm(nm, Name, Value, ...)
```

Builds the same Lorentzian-broadened IR/Raman continuum spectra as `G09_spectra`/`G16_spectra`,

but directly from an already-parsed `nm` (normal modes) struct instead of reading a `.log`/`.out` file. Accepts the `nm` struct returned by `G09_nmodes`/`G16_nmodes`, or the `.nm` sub-struct returned by `G09_fchk_read`/`G16_fchk_read` (Section 6.1) — all four share the same field layout (`.freq`, `.IR`, `.Raman`, `.has_Raman`, `.Nmodes`, `.filename`), so a single function works with any of them regardless of Gaussian version or data source.

> This fills a gap left by `G09_fchk_read`/`G16_fchk_read`: their output struct's `.nm` field has frequencies and IR/Raman intensities, but — unlike `G09_spectra`/`G16_spectra` — does not itself build the broadened continuum spectrum. `G_spectra_nm` does exactly that, directly from `data.nm`, with no `.out`/`.log` file needed at all.

Option	Default	Description
`'FWHM'`	`10`	Lorentzian FWHM (cm^{-1})
`'xmin'`	`0`	Lower wavenumber limit (cm^{-1})
`'xmax'`	`4000`	Upper wavenumber limit (cm^{-1})
`'dx'`	`1`	Grid step (cm^{-1})
`'normalize'`	`false`	Normalise continua to maximum = 1
`'plot'`	`false`	Generate figure

Output fields are identical to `G09_spectra`/`G16_spectra`'s (Section 4.8 above): `.freq`, `.IR`, `.Raman`, `.has_Raman`, `.Nmodes`, `.x`, `.IR_cont`, `.Raman_cont`, `.FWHM`, `.filename`.

```
data = G16_fchk_read('molecule.fchk');
sp   = G_spectra_nm(data.nm, 'FWHM', 15, 'plot', true);   % from a .
   fchk, no .out needed

nm = G16_nmodes('molecule.out');
sp = G_spectra_nm(nm, 'xmin', 400, 'normalize', true);    % from an
   already-read .out
```

5 Atomic charges

5.1 G09_charges / G16_charges

```
ch = G09_charges(filename)
ch = G09_charges(filename, 'type', 'APT')
ch = G09_charges(filename, 'mode', 'heavy')
ch = G09_charges(filename, 'plot', true, 'threshold', 0.05)
```

Extracts Mulliken or APT atomic charges and optionally renders them on the 3D structure, with an optional dipole-moment arrow overlay.

> **Robustness:** handles all known Mulliken label variants across Gaussian revisions/OSes (e.g. `"Mulliken atomic charges:"` on G09 Rev. A.02/Windows and some Linux builds; `"Mulliken charges:"` on G09 Rev. B.01/C.01/D.01 and on all G16 revisions), by matching any line containing the charge-type keyword and `"charges:"`, excluding `"Sum of"` and H-summed lines. The exact label found is reported in `ch.label`.

Option	Default	Description
`'type'`	`'Mulliken'`	`'Mulliken'` | `'APT'`
`'mode'`	`'atom'`	`'atom'` | `'heavy'` (H charges summed onto heavy atoms)
`'plot'`	`true`	Render labels on the 3D structure
`'AtomScale'`	`0.35`	CPK sphere scale
`'BondTol'`	`1.30`	Bond detection tolerance
`'BondList'`	`[]`	Passed straight through to `draw_molecule`: explicit atom-index pairs (optionally with a pre-computed bond order), bypassing `BondTol` — see the note below
`'FontSize'`	`8`	Charge-label font size
`'ColorScale'`	`'RdBu'`	`'RdBu'` (blue=neg/red=pos) | `'none'`
`'threshold'`	`0`	Hide labels with $\|q\| <$ threshold
`'ShowDipole'`	`false`	Overlay the total dipole moment vector *in the 3D plot*. The dipole moment itself (`.dipole` and related output fields, below) is always computed regardless of this option — see the note below
`'DipoleOrigin'`	`'negcharge'`	Arrow anchor: `'negcharge'` (centroid of negative charges) | `'poscharge'` | `'centroid'` | `[x y z]`
`'DipoleScale'`	`1.0`	Arrow length, Å per Debye
`'DipoleColor'`	`[0 0.6 0]`	Arrow colour
`'DipoleLineWidth'`	`2.5`	Arrow line width
`'ShowDipoleLabel'`	`true`	Annotate the arrow with $\|\mu\|$
`'DipoleFontSize'`	`11`	Dipole label font size
`'DipoleUnits'`	`'Debye'`	Units for $\|\mu\|$: `'Debye'` | `'au'`
`'Lines'`	`{}`	Pre-read file lines

Field	Description
`.symbols`, `.charges`	Atomic symbols, per-atom charges (e)
`.charges_H`	H-summed charges on heavy atoms
`.sum_q`	Sum of all charges (≈ 0 for neutral molecules)
`.type`, `.label`	`'Mulliken'`/`'APT'`, exact header label found
`.Natoms`	
`.dipole`	`[1x3]` dipole moment (Debye) — always computed (see the note below); `[]` if the file has no readable dipole moment
`.dipole_origin`	`[1x3]` anchor point used for the arrow (Å), computed alongside `.dipole`
`.dipole_Debye`	$\|\mu\|$ in Debye, computed alongside `.dipole`
`.dipole_au`	$\|\mu\|$ in atomic units, computed alongside `.dipole`

The dipole moment is always computed. `'ShowDipole'` only controls whether the arrow is actually drawn in the 3D plot (and, when true, whether a dipole read failure is reported as a warning) — `.dipole`/`.dipole_origin`/`.dipole_Debye`/ `.dipole_au` are populated in the returned struct regardless, since reading the dipole moment is cheap. The underlying `G09/G16_dipole_polar` call's own console summary is suppressed unless `'ShowDipole'` is true, so this does not add unrequested console output to the common (`'ShowDipole'` left at its default `false`) case.

Rendering NBO-derived bond orders. `'BondList'` accepts the same `[Nbonds x 3]` `Atom1`, `Atom2`, `BondOrder` matrix that `draw_molecule` does (Section 13.4), so the charge overlay renders on top of the real NBO-derived bond order instead of the geometric estimate:

```
bt = G16_nbo_bonds('molecule_nbo.out');
bond_list = [bt.Atom1, bt.Atom2, double(bt.BondOrder)];
G16_charges('molecule_nbo.out', 'BondList', bond_list, 'ShowDipole
    ', true);
```

5.2 G09_charges_fchk / G16_charges_fchk

```
ch_out = G09_charges_fchk(mol, ch)
ch_out = G09_charges_fchk(mol, ch, Name, Value, ...)

ch_out = G16_charges_fchk(mol, ch)
```

Visualises atomic charges directly from a `G09_fchk_read`/ `G16_fchk_read` struct, without requiring the corresponding `.log`/`.out` file. Mirrors the interface of `G09_charges`/ `G16_charges` exactly, so the two can be used interchangeably once `data.mol` and `data.ch` are available.

Option	Default	Description
'mode'	'atom'	'atom' | 'heavy' (requires ch.charges_H non-empty — .fchk files have no native H-summed data; use G09_charges/G16_charges on the .log for that)
'plot'	true	
'AtomScale'	0.35	
'BondTol'	1.30	
'FontSize'	8	
'ColorScale'	'RdBu'	'RdBu' | 'none'
'threshold'	0	

```
data = G16_fchk_read('3typ.fchk');
ch   = G16_charges_fchk(data.mol, data.ch);                        %
   default
ch   = G16_charges_fchk(data.mol, data.ch, 'plot', false, 'threshold',
    0.05);
ch   = G16_charges_fchk(data.mol, data.ch, 'ColorScale', 'none'); %
   drop-in for G16_charges
```

Requires mol with fields .symbols, .xyz, .Natoms, .filename, and ch with fields .charges, .symbols, .type, .Natoms (.charges_H optional).

> Since the .fchk format is identical for G09 and G16, G09_charges_fchk and G16_charges_fchk are functionally identical (the latter simply calls G16_draw_molecule internally instead of G09_draw_molecule) — both are provided so that either toolbox folder alone is enough.

6 Formatted checkpoint (.fchk) support

6.1 G09_fchk_read / G16_fchk_read

```
data = G09_fchk_read(filename)
data = G09_fchk_read(filename, 'verbose', false)

data = G16_fchk_read(filename)
```

Reads a Gaussian 09/16 formatted checkpoint file (`.fchk`), generated with `formchk jobname.chk jobname.fchk`. Uses a single generic section parser (no hard-coded line counts) and is compatible with both G09 and G16 `.fchk` files.

> Supersedes the old `.fck`/newzmat-format reader: reads the modern `.fchk` ASCII format, returns SI-ready fields (XYZ in Å, dipole in Debye), and needs no external helper functions. The formatted-checkpoint layout does not depend on the Gaussian version, so `G09_fchk_read` and `G16_fchk_read` are identical and each works on `.fchk` files produced by *either* G09 or G16 — both are provided purely so that either toolbox folder alone (`G09/` or `G16/`) is enough to read any `.fchk` file, without needing the other one on the path too.

Option	Default	Description
`'verbose'`	`true`	Print progress messages while parsing

Output struct `data` (grouped by section):

Field	Description
`.title`, `.method`, `.basis`	First line, method (e.g. `'RB3LYP'`), basis set (e.g. `'def2SVPP'`)
`.Nat`, `.charge`, `.mult`	Number of atoms, molecular charge, spin multiplicity
`.Nelec`, `.Nalpha`, `.Nbeta`	Total, alpha, beta electron counts
`.Nbasis`, `.Nbasis_indep`	Basis functions, independent basis functions
`.symbols`, `.AN`, `.masses`	Atomic symbols, atomic numbers, real atomic masses (AMU)
`.xyz`, `.xyz_bohr`	Coordinates (Å), coordinates (Bohr)
`.SCF_energy`, `.total_energy`, `.virial_ratio`	SCF energy, total energy (Hartree), virial ratio (≈ 2)
`.alpha_orb_energies`, `.beta_orb_energies`	MO energies (Hartree); beta is `[]` if RHF
`.alpha_MO_coeff`	Alpha MO coefficients (column-major)
`.mulliken_charges`	Mulliken charges (e)
`.HOMO_idx`, `.HOMO_eV`, `.LUMO_eV`, `.gap_eV`	Frontier-orbital summary
`.gradient`, `.rms_force`	Cartesian gradient (Hartree/Bohr), RMS force

7 Molecule and normal-mode visualisation

7.1 G09_draw_molecule / G16_draw_molecule

```
G09_draw_molecule(mol)
G09_draw_molecule(mol, 'Name', Value, ...)
```

Renders a 3D CPK ball-and-stick model from a `mol` struct (as returned by `G09_structure`/`G16_structure`), with bonds detected automatically from a covalent-radius criterion and drawn as 1, 2, or 3 parallel lines according to an estimated bond order.

Option	Default	Description
`'AtomScale'`	`0.35`	Sphere radius as a fraction of the covalent radius
`'BondTol'`	`1.30`	Bond tolerance: bonded if $d < (r_1 + r_2) \times$ `BondTol`
`'ShowLabels'`	`true`	Show atom labels (`"C1"`, `"H2"`, ...)
`'ShowLegend'`	`true`	Show the element-colour legend
`'Title'`	filename	Figure title
`'BgColor'`	`[0.95 0.95 0.95]`	Axes background colour
`'Ax'`	(new figure)	Draw into an existing axes handle
`'ShowAxes'`	`false`	Draw a small Cartesian X/Y/Z reference triad, anchored at the lower-left-front corner of the plotted molecule
`'AxesLength'`	`[]`	Length of the X/Y/Z arrows, in Å (`[]` = auto, 20% of the molecule's bounding-box diagonal)
`'BondList'`	`[]`	$[N_{\text{bonds}} \times 2]$ or $[N_{\text{bonds}} \times 3]$ explicit atom-index pairs (optionally with a 3rd column giving a pre-computed bond order 1/2/3) to draw as bonds, bypassing the `BondTol` distance criterion (`[]` = auto-detect). Used by `G09_animate_mode`/`G16_animate_mode` to keep a fixed bond topology (and, with the 3rd column, a fixed bond order) across all frames of an animation

```
mol = G09_structure('zeatin.out');
G09_draw_molecule(mol);
G09_draw_molecule(mol, 'ShowLabels', false, 'AtomScale', 0.4);
G09_draw_molecule(mol, 'ShowAxes', true);
```

Bond order (single/double/triple). For C-C and C-O pairs, bond order is estimated purely from bond length (any other element pair, including C-N, is always drawn as a single bond) and rendered as 1/2/3 parallel lines in the usual chemical-drawing convention — a geometric estimate, not Gaussian's own bond-order analysis (e.g. Wiberg/NBO indices). For an actual bond order derived from Gaussian's own NBO analysis, see `G09_nbo_bonds`/`G16_nbo_bonds` (Section 13.4). The C-C double/single threshold is deliberately set to 1.36 Å rather than the generic reference-length midpoint (≈1.44 Å): symmetric aromatic rings have essentially equal C-C bond lengths (≈1.39–1.40 Å, with no real single/-double alternation to recover from geometry alone), so the generic midpoint would classify every ring bond as double. With the adjusted threshold, aromatic rings are instead rendered as all-single — a more honest depiction than an arbitrary alternating pattern, since the true bond order is ≈1.5 all the way around the ring.

Rendering NBO-derived bond orders. `nbo_bonds`'s output table can be fed straight into `draw_molecule`'s `'BondList'` parameter (Section 13.4), replacing the geometric estimate above with the real bond order from Gaussian's own NBO analysis:

```
mol = G16_structure('molecule_nbo.out');
bt  = G16_nbo_bonds('molecule_nbo.out');   % requires 'pop=nbo' in
   the source job

bond_list = [bt.Atom1, bt.Atom2, double(bt.BondOrder)];
G16_draw_molecule(mol, 'BondList', bond_list);
```

Why C-N is excluded. C-N was classified by length in an earlier version of the toolbox, using the same kind of threshold as C-C/C-O. Unlike C-C, a single threshold cannot separate aromatic C-N (e.g. pyridine rings, ≈1.34 Å) from a genuine C=N: on asymmetric pyridine-like rings this produced one ring C-N bond rendered double and its chemically-equivalent neighbour rendered single, an inconsistent and misleading picture. C-N is therefore always drawn as a single bond, regardless of length.

Bug fixed (2026-08-05): lighting not following interactive rotation. The plot supports interactive rotation via `rotate3d` (drag with the mouse). `CAMLIGHT` only positions a light relative to the camera at the moment it is called — MATLAB's own documentation is explicit about this: "In order for a light created with CAMLIGHT to stay in a constant position relative to the camera, CAMLIGHT must be called whenever the camera is moved." `ROTATE3D` does not do this on its own, so previously the two lights stayed fixed in the axes' 3-D world frame while the molecule rotated underneath them: after a large enough rotation (e.g. 180 degrees) the CPK spheres ended up lit from behind instead of from the viewer's side, losing the shading gradient that gives them their 3-D appearance and making them look flat/uniformly lit. Fixed by keeping a handle to both lights and assigning `rotate3d`'s `ActionPostCallback` property (not an event — assigning it again simply replaces the previous callback, so repeated calls into the same figure, e.g. one `draw_molecule` call per `animate_mode` frame, do not accumulate callbacks) to reposition both lights via `CAMLIGHT` after every interactive rotation.

No differences between the G09 and G16 versions beyond the input struct's origin.

7.2 G09_draw_mode / G16_draw_mode

```
G09_draw_mode(mol, nm, mode_idx)
G09_draw_mode(mol, nm, mode_idx, 'Name', Value, ...)
```

Renders a single vibrational normal mode as displacement arrows over the CPK structure.

Option	Default	Description
`'Scale'`	`1.5`	Arrow length scale
`'ArrowColor'`	`[1 0.4 0.1]`	Arrow colour
`'AtomScale'`	`0.35`	CPK sphere scale factor
`'BondTol'`	`1.30`	Bond detection tolerance
`'ShowLabels'`	`false`	Show atom index labels
`'FlipSign'`	`false`	Invert all arrow directions

Why `'FlipSign'` exists. Normal-mode eigenvectors have an arbitrary overall sign — a mode and its 180°-phase-shifted twin are physically identical. If the arrows point opposite to GaussView's rendering for a given mode, set `'FlipSign', true` to flip them. This is purely cosmetic and has no effect on frequencies or intensities.

7.3 G09_modeViewer / G16_modeViewer

```
G09_modeViewer(filename)
G09_modeViewer(filename, 'Name', Value, ...)
```

Interactive selector GUI for browsing vibrational modes. Reads the structure and all normal modes via `G09_structure`/`G09_nmodes`, then opens a window listing every mode (index, frequency, symmetry). Selecting an entry calls `G09_draw_mode` to render that mode in a new figure window; previously drawn mode figures stay open, so different modes (or render settings) can be compared side by side.

Window controls:

- *Order by:* sort the mode list by mode number (default), IR intensity, or Raman intensity (descending; only offered when Raman data is present). The current selection is preserved across reordering.
- A text box to set a custom title on the current mode figure.
- A control for every `G09_draw_mode` option (`Scale`, `ArrowColor`, `AtomScale`, `BondTol`, `ShowLabels`, `FlipSign`); changing any of them immediately redraws the selected mode in a new figure. Arrow colour can be set from a named preset (Orange/Red/Blue/Green/Black/ Magenta) or a custom colour.
- A *Save figure...* button exporting the target mode figure to JPEG, EPS, or PDF (EPS/PDF as true vector graphics).
- An *Animate mode (MP4)...* button exporting an MP4 animation of the currently selected mode via `G09_animate_mode`/ `G16_animate_mode`, using the current Scale/AtomScale/ BondTol/ShowLabels/FlipSign settings and, automatically, the camera orientation of the currently displayed mode figure (a manual rotation carries over into the animation).

Name-Value arguments passed to `G09_modeViewer` pre-populate the corresponding option control(s), e.g. `G09_modeViewer('file.log', 'Scale', 2, 'ShowLabels', true)`.

```
G09_modeViewer('V_E00t.out');
G16_modeViewer('INDACO_E025_ppDP.LOG', 'Scale', 2, 'ShowLabels', true)
   ;
```

See also `G09_structure`/`G16_structure`, `G09_nmodes`/`G16_nmodes`, `G09_draw_mode`/`G16_draw_mode`, `G09_animate_mode`/`G16_animate_mode`.

7.4 G09_animate_mode / G16_animate_mode

```
outfile = G09_animate_mode(mol, nm, mode_idx)
outfile = G09_animate_mode(mol, nm, mode_idx, 'Name', Value, ...)

outfile = G16_animate_mode(mol, nm, mode_idx)
```

Exports an MP4 animation of a single vibrational mode: the molecule oscillates along the mode's displacement vector (equilibrium ± amplitude), in the style of GaussView's mode animations, and the result is saved via `VideoWriter` (`'MPEG-4'` profile). A figure window opens and animates

live while the video is rendered; avoid interacting with it until the function returns.

Option	Default	Description
'Filename'	[] (auto)	Output path; default <source>_mode<N>.mp4, .mp4 appended if missing
'Scale'	1.5	Displacement amplitude scale, same meaning as G09_draw_mode/G16_draw_mode's 'Scale'
'FlipSign'	false	Invert the displacement direction
'AtomScale'	0.35	See G09_draw_molecule/G16_draw_molecule
'BondTol'	1.30	See G09_draw_molecule/G16_draw_molecule; also used to compute the fixed bond list (see below)
'ShowLabels'	false	See G09_draw_molecule/G16_draw_molecule
'FramesPerCycle'	30	Frames per oscillation period
'NCycles'	2	Number of periods rendered
'FPS'	20	Video frame rate
'View'	[]	[azimuth elevation] in degrees, the starting camera orientation ([] = MATLAB's default 3D view, azimuth= −37.5, elevation= 30). Pass the output of MATLAB's own view(ax) to match an already-rotated figure
'ShowWaitbar'	true	Print rendering progress to the command window
'Crop'	true	Tightly crop every frame around the molecule instead of writing the whole (mostly blank) figure window (see below)
'CropMargin'	12	Blank margin kept around the molecule when 'Crop' is true, in pixels
'ShowTitle'	false	Draw the filename/mode/frequency title text in every frame. Left off by default so the title's own bounding box does not force a taller crop than the molecule itself needs

```
mol = G16_structure('V_E00t.out');
nm  = G16_nmodes('V_E00t.out');
G16_animate_mode(mol, nm, 70, 'Scale', 2, 'FPS', 25);
```

Fixed bond list and bond order. Bonds (and their estimated order, 1/2/3) are computed once from the *equilibrium* geometry (via G09_get_bond_length/G16_get_bond_length, using 'BondTol' as the tolerance) and passed to G09_draw_molecule/G16_draw_molecule through its 'BondList' parameter (atom-index pairs plus a 3rd bond-order column), instead of being re-detected from instantaneous inter-atomic distances on every frame. Without this, bonds would flicker in and out — and double/triple bonds would flicker to/from single — as oscillating atoms cross the relevant distance thresholds.

No graphical waitbar. Rendering progress is printed to the command window rather than shown with MATLAB's waitbar: on some MATLAB versions, waitbar itself was found to error intermittently across many rapid updates ("Not enough input arguments" inside its own internal title() call).

Axes handling. Each frame's axes are deleted and recreated (rather than cleared with `cla`) before redrawing: in this 3D-plus-lighting configuration, `cla(ax)` was found to leave stale graphics objects behind, causing the exported video to show all frames superimposed instead of one at a time.
Frame cropping. Without `'Crop'`, every frame is the full figure window (default MATLAB figure size), leaving most of it blank around the molecule. The crop rectangle is computed *once*, from the two oscillation extremes (phase +1 and −1, which bound the whole animation), by finding the tight bounding box of non-background pixels and padding it by `'CropMargin'`; the same rectangle is then applied to every frame. Because the axis limits are already fixed across the whole animation (see above), this rectangle stays valid throughout the video without ever clipping an atom at any phase, typically reducing the frame area by 70–80%.

No differences between the G09 and G16 versions beyond the input structs' origin.

See also `G09_draw_molecule/G16_draw_molecule`, `G09_draw_mode/G16_draw_mode`, `G09_modeViewer/G16_modeViewer`, `G09_get_bond_length/G16_get_bond_length`.

7.5 G09_draw_deformation / G16_draw_deformation

```
G09_draw_deformation(mol1, mol2)
G09_draw_deformation(mol1, mol2, 'Name', Value, ...)
```

Visualises the geometric deformation between two related structures of the *same* molecule — typically with vs without a small static electric field (finite-field NLO workflows), two conformers, or before/after optimisation — by drawing an arrow from each atom's position in `mol1` to its position in `mol2` (`mol2.xyz - mol1.xyz`), the same displacement-arrow style `G09_draw_mode` uses for a normal mode, but here the actual geometric shift between two structures rather than a vibrational eigenvector. `mol1`/`mol2` can come from `G09_structure/G16_structure`, `G_read_structure_file`, or `G09_fchk_read/G16_fchk_read`'s `.mol` sub-struct — same atom count and ordering required.

Option	Default	Description
`'Overlay'`	`false`	See "Two display modes" below
`'Scale'`	`1`	Arrow length AND overlay-position multiplier, applied identically to both (see below)
`'ArrowColor'`	`[1 0.4 0.1]`	Arrow colour (same default as `G09_draw_mode`)
`'Overlay2Color'`	`[0.75 0.1 0.1]`	Colour of `mol2`'s skeletal overlay when `'Overlay'` is true
`'AtomScale'`	`0.35`	CPK sphere scale for `mol1`
`'BondTol'`	`1.30`	Bond-detection tolerance, both structures
`'ShowLabels'`	`false`	Show atom index labels on `mol1`
`'ArrowThreshold'`	`0.05`	Skip the arrow for atoms whose displacement is below this fraction of the single largest per-atom displacement

Two display modes

- `'Overlay', false` (default): draws `mol1` only, as a full CPK ball-and-stick model, with displacement arrows.

- `'Overlay', true`: ALSO draws `mol2` as a simplified skeletal overlay (thin dashed lines + small dots, no spheres) in `'Overlay2Color'`, so both ends of every arrow are visually anchored to a real atom position instead of just an arrowhead in empty space.

Console report. Every call prints the largest single-atom displacement (with which atom) and the RMSD over all atoms, in Å — the numbers to look at before picking `'Scale'`: real deformations (e.g. field-induced) are often only 10^{-3}–10^{-2} Å, far too small to see at `Scale=1`.
Why `'Scale'` affects the overlay too. The overlay is drawn at `mol1.xyz + Scale*(mol2.xyz - mol1.xyz)`, i.e. the SAME exaggerated position the arrows point to — *not* `mol2`'s true coordinates. At `Scale` $\neq 1$ (the normal case for these typically-tiny deformations), drawing the overlay at the true, unscaled position would leave it visually detached from the arrow tips, since the arrows themselves are drawn exaggerated. Bond connectivity for the overlay is still detected from `mol2`'s true, unscaled geometry (an exaggerated geometry at a large `Scale` is not a real molecular geometry and would misjudge bonding distances).

```
mol1 = G16_structure('nofield.out');
mol2 = G16_structure('field_x025.out');
G16_draw_deformation(mol1, mol2, 'Scale', 200);
G16_draw_deformation(mol1, mol2, 'Overlay', true, 'Scale', 200);
```

No differences between the G09 and G16 versions beyond which `draw_molecule`/`get_bond_length` pair each calls internally.

See also `G09_draw_molecule`/`G16_draw_molecule`, `G09_draw_mode`/`G16_draw_mode`, `G09_get_bond_length`/`G16_get_bond_length`.

8 au2SI: atomic units ↔ SI conversions

`AU_convert/AU_table` convert physical quantities between atomic units (au) and SI (or other common units used in computational chemistry); `G_gaussian_field_convert` additionally converts Gaussian's own `Field=X+N` route-keyword integer to/from a physical field strength. All conversion factors are based on CODATA 2022 (NIST, November 2024 release). The toolbox covers 16 physical-quantity categories, all three functions are also ported to Python directly inside `G16parser` (§8.6):

Quantity	**au unit**
Energy	Hartree (E_h)
Length	Bohr (a_0)
Dipole moment	$e \cdot a_0$
Polarisability α	$e^2 a_0^2 / E_h$
1st hyperpolarisability β	$e^3 a_0^3 / E_h^2$
2nd hyperpolarisability γ	$e^4 a_0^4 / E_h^3$
Force	E_h / a_0
Force constant	E_h / a_0^2
Frequency	$E_h / \hbar$
Time	$\hbar / E_h$
Pressure	E_h / a_0^3
Electric field	$E_h / (e\, a_0)$
Mass	m_e
Charge	e
Magnetic moment	μ_B
Temperature	$k_B T \leftrightarrow K$

> No additional toolboxes or external dependencies are required. Both functions use only MATLAB built-ins.

8.1 AU_convert

```
result = AU_convert(value, quantity, direction)
result = AU_convert(value, quantity, direction, 'target', unit)
AU_convert([], 'help', '')                % print full reference table
```

Argument	**Description**
`value`	Scalar or array of values to convert (double). Arrays are converted element-wise.
`quantity`	Physical quantity string (case-insensitive) — see the reference table below.
`direction`	`'au2si'` — from atomic units to SI (or target unit); `'si2au'` — from SI (or target unit) to atomic units
`'target'`	Name-Value: target unit when multiple options exist for the SI side. Optional — a default is used when omitted.

`AU_convert` returns a `double` array of the same shape as `value`. When `value` is a scalar, the

conversion is also printed to the console.

> **Quantity strings and available targets.** The table below lists every supported quantity string, the atomic unit on the left side, and the available target units on the SI side. The default target (used when `'target'` is omitted) is the first row of each quantity block.

quantity string	au unit	'target'	SI/target unit (1 au =)
`'energy'`	Hartree (E_h)	(default)	J $4.3597447 \times 10^{-18}$
		`'kJ/mol'`	2625.4996 kJ/mol
		`'kcal/mol'`	627.5095 kcal/mol
		`'eV'`	27.211386 eV
		`'cm-1'`	219474.631 cm^{-1}
`'length'`	Bohr (a_0)	(default)	m $5.2917721 \times 10^{-11}$
		`'Angstrom'`	0.5291772 Å
		`'pm'`	52.9177 pm
`'dipole'`	$e \cdot a_0$	(default)	C·m $8.4783536 \times 10^{-30}$
		`'Debye'`	2.541747 D
`'polar'`	$e^2a_0^2/E_h$	(default)	$\mathrm{C}^2\cdot\mathrm{m}^2\cdot\mathrm{J}^{-1}$ $1.6487773 \times 10^{-41}$
		`'Angstrom3'`	0.148185 Å^3 (volume)
		`'esu'`	1.48185×10^{-25} cm^3 (esu)
`'beta'`	$e^3a_0^3/E_h^2$	(default)	$\mathrm{C}^3\cdot\mathrm{m}^3\cdot\mathrm{J}^{-2}$ 3.2064×10^{-53}
		`'esu'`	8.6392×10^{-33} esu
`'gamma'`	$e^4a_0^4/E_h^3$	(default)	$\mathrm{C}^4\cdot\mathrm{m}^4\cdot\mathrm{J}^{-3}$ 6.2354×10^{-65}
`'force'`	E_h/a_0	(default)	N 8.2387×10^{-8}
		`'nN'`	82.387 nN
`'frcconst'`	E_h/a_0^2	(default)	N/m 1556.893
		`'mDyne/A'`	15.5689 mDyne/Å
`'frequency'`	$E_h/\hbar$	(default)	rad/s 4.13414×10^{16}
		`'Hz'`	6.57968×10^{15} Hz
		`'THz'`	6579.684 THz
		`'cm-1'`	219474.631 cm^{-1}
`'time'`	$\hbar/E_h$	(default)	s 2.41888×10^{-17}
		`'fs'`	0.024189 fs
		`'as'`	24.1888 as
`'pressure'`	E_h/a_0^3	(default)	Pa 2.94210×10^{13}
		`'GPa'`	29421.0 GPa
		`'atm'`	2.90443×10^{8} atm
`'efield'`	$E_h/(e\,a_0)$	(default)	V/m 5.14221×10^{11}
		`'GV/m'`	514.221 GV/m
		`'V/A'`	51.4221 V/Å
`'mass'`	m_e	(default)	kg 9.10938×10^{-31}
		`'Da'`	5.48580×10^{-4} Da (AMU)
`'charge'`	e	(default)	C 1.60218×10^{-19}
`'magmom'`	μ_B	(default)	J/T 9.27401×10^{-24}
`'temp'`	$E_h \leftrightarrow$ K	(default)	K (via k_BT) 315775.0

Examples

```
% SCF energy: au -> kJ/mol and eV
```

```
AU_convert(-875.9322, 'energy', 'au2si', 'target', 'kJ/mol')
AU_convert(-875.9322, 'energy', 'au2si', 'target', 'eV')

% Relative Gibbs energy between two conformers
dG_au = en2.G - en1.G;
dG_kJ = AU_convert(dG_au, 'energy', 'au2si', 'target', 'kJ/mol');

% Vibrational zero-point energy (freq in cm-1 -> Eh)
zpe_au = AU_convert(1582.8/2, 'energy', 'si2au', 'target', 'cm-1');

% Dipole moment: au -> Debye (G16_dipole_polar with 'units','au')
dp   = G16_dipole_polar('V_E00t.out', 'units', 'au');
mu_D = AU_convert(dp.mu_tot, 'dipole', 'au2si', 'target', 'Debye');

% Polarisability tensor: au -> Angstrom^3 (volume polarisability)
dp       = G09_dipole_polar('INDACO_E025_ppDP.LOG');
alpha_A3 = AU_convert(dp.alpha_tensor, 'polar', 'au2si', 'target', '
   Angstrom3');

% Force constants: au -> mDyne/Angstrom
nm       = G16_nmodes('V_E00t.out');
fc_mdyne = AU_convert(nm.frcconst, 'frcconst', 'au2si', 'target', '
   mDyne/A');

% Geometry array: Angstrom -> Bohr
mol      = G16_structure('V_E00t.out');
xyz_bohr = AU_convert(mol.xyz, 'length', 'si2au', 'target', 'Angstrom
   ');

% Thermal energy kBT at room temperature
kBT = AU_convert(298.15, 'temp', 'si2au');

% Hyperpolarisability: au -> esu
hp       = G16_hyperpolar('V_E00t.out');
beta_esu = AU_convert(hp.beta0.beta_vec, 'beta', 'au2si', 'target', '
   esu');
```

8.2 AU_table

```
AU_table                  % print complete table
AU_table('energy')        % print energy section only
AU_table('constants')     % print fundamental constants section
AU_table('polar')         % print polarisability section
```

Prints a formatted reference table of conversion factors and fundamental physical constants to the MATLAB console. The filter argument is case-insensitive and accepts the same quantity strings as `AU_convert` (`energy`, `length`, `dipole`, `polar`, `beta`, `gamma`, `force`, `frcconst`, `frequency`, `time`, `pressure`, `efield`, `mass`, `charge`, `magmom`, `temp`, plus `constants` for the fundamental-constants section).

```
>> AU_table('energy')

  -- ENERGY --
  1 Hartree (Eh)              = 4.3597447222e-18 J
                                = 2625.499639 kJ/mol
                                = 627.509474 kcal/mol
                                = 27.211386 eV
```

```
                                  = 219474.6314 cm-1
                                  = 27211.386 meV
  1 eV                           = 1.6021766340e-19 J
                                  = 96.4853 kJ/mol
                                  = 8065.5439 cm-1
  1 cm-1                         = 1.9864459e-23 J
                                  = 4.55634e-06 Hartree
  1 kJ/mol                       = 3.80880e-04 Hartree
  1 kcal/mol                     = 1.59360e-03 Hartree
  1 kBT (298.15K)                = 9.44185e-04 Hartree  (= 2.4790 kJ/mol)
```

8.3 G_gaussian_field_convert

```
out = G_gaussian_field_convert(N, 'g2phys')
out = G_gaussian_field_convert(N, 'g2phys', 'Unit', 'V/Ang')
out = G_gaussian_field_convert(value, 'phys2g', 'Unit', 'au')
```

Converts between Gaussian's `Field=X+N` route-keyword integer and a physical electric-field strength. Gaussian's `Field` keyword (e.g. `Field=X+10`, `Field=X-5`) specifies a static electric field whose magnitude, in atomic units, is the keyword's integer N times 0.0001 — i.e. $N \times 10^{-4}$ a.u.

Verified against the source, not assumed. This convention was confirmed directly from Gaussian's own keyword reference (https://gaussian.com/field/): *"N*0.0001 specifies the magnitude of the field in atomic units"* (for the $M \pm N$ format, the one a plain dipole field like `Field=X+10` uses) — quoted from the source rather than assumed, since this exact convention is easy to get wrong (a field-magnitude keyword invites exactly this kind of silent off-by-a-power-of-ten mistake).

Argument	Description
`value`	Gaussian keyword integer N (direction `'g2phys'`) or a physical field strength (direction `'phys2g'`); may be a numeric array (applied elementwise)
`direction`	`'g2phys'` — $N \rightarrow$ physical field strength; `'phys2g'` — physical field strength $\rightarrow N$

Option	Default	Description
`'Unit'`	`'au'`	`'au'` | `'V/m'` | `'V/cm'` | `'V/Ang'` — the physical unit on the non-Gaussian side, for either direction
`'Print'`	`true`	Print a one-line summary

Rounding on `'phys2g'`. Gaussian's keyword can only express multiples of 0.0001 a.u., so converting an arbitrary physical value back to N necessarily rounds to the nearest integer. A warning is printed whenever that rounding changes the value by more than 0.5%, so a value that does not correspond to a "clean" N is never silently misrepresented.
Consistency with `AU_convert`. The atomic unit of electric field ($E_h/(e\,a_0)$) is computed from the same CODATA 2022 constants `AU_convert` itself uses (Section 8.1), not a separately rounded literal — cross-checked to match `AU_convert(..., 'efield', ..., 'target', 'V/A')` exactly.

```
G_gaussian_field_convert(10, 'g2phys')                    % N=10 ->
    0.0010 au
G_gaussian_field_convert(10, 'g2phys', 'Unit', 'V/Ang')   % N=10 ->
    0.0514 V/Ang
G_gaussian_field_convert(0.0025, 'phys2g', 'Unit', 'au')  % 0.0025 au
    -> N=25
```

See also `AU_convert` (Section 8.1).

8.4 Integration with the G09/G16 Toolbox

`AU_convert` operates on plain numeric arrays and is fully compatible with all structs returned by `G09_*/G16_*` functions. The table below lists the most common conversions needed after parsing a Gaussian output file.

Field	Quantity / target	Typical use
`en.SCF` / `en.G` / `en.H`	`'energy'` → `'kJ/mol'`	Relative energies, reaction profiles
`en.SCF`	`'energy'` → `'eV'`	Band gaps, ionisation potentials
`sp.freq`	already cm^{-1}	No conversion needed
`nm.frcconst`	`'frcconst'` → `'mDyne/A'`	Comparison with experimental force constants
`mol.xyz`	already Å	Convert to Bohr (`'length'`, `si2au`) only for QM codes that need it
`dp.mu_tot`	`'dipole'` → `'Debye'`	Dipole magnitude (au by default)
`dp.alpha_tensor`	`'polar'` → `'Angstrom3'`	Volume polarisability for SERS/NLO analysis
`hp.beta0.beta_vec`	`'beta'` → `'esu'`	Hyperpolarisability for NLO comparison
$k_B T$ at T	`'temp'`, `si2au`	Boltzmann weights in conformer analysis
`en.S_JmolK`	already $\mathrm{J\ mol^{-1}\ K^{-1}}$	No conversion needed

> Fields already returned in physical (non-au) units by G09/G16 Toolbox functions: `sp.freq` (cm^{-1}), `en.S_JmolK` ($\mathrm{J\ mol^{-1}\ K^{-1}}$), `en.ZPE_kJ` (kJ/mol), `mol.xyz` (Å), `sp.IR` (KM/Mole), `sp.Raman` ($\mathrm{Å}^4$/AMU). These do not require `AU_convert`.

Worked example: NLO property analysis

```
% Load properties from G16 output
dp = G16_dipole_polar('V_E00t.out');
hp = G16_hyperpolar('V_E00t.out');

% Convert to common NLO units
mu_D     = AU_convert(dp.mu_tot,          'dipole', 'au2si', 'target',
    'Debye');
```

```
alpha_A3 = AU_convert(dp.alpha_iso,      'polar',  'au2si', 'target',
    'Angstrom3');
beta_esu = AU_convert(hp.beta0.beta_vec, 'beta',   'au2si', 'target',
    'esu');

fprintf('mu      = %.4f D\n',   mu_D)
fprintf('alpha   = %.3f A^3\n', alpha_A3)
fprintf('|beta|  = %.3e esu\n', beta_esu)
```

Worked example: Boltzmann-weighted conformer ensemble

```
T   = 298.15;                                % K
kBT = AU_convert(T, 'temp', 'si2au');        % Eh

files = {'conf1.out', 'conf2.out', 'conf3.out'};
G = zeros(1, numel(files));
for i = 1:numel(files)
    en   = G16_energy(files{i});
    G(i) = en.G;
end
dG    = G - min(G);                          % relative Gibbs energy (Eh)
w     = exp(-dG / kBT);                      % Boltzmann weights
w     = w / sum(w);                          % normalised
dG_kJ = AU_convert(dG, 'energy', 'au2si', 'target', 'kJ/mol');
```

8.5 CODATA 2022 fundamental constants

All conversion factors in `AU_convert`/`AU_table` are derived from the CODATA 2022 values below. Values marked *(exact)* are fixed by definition in the 2019 SI redefinition.

Constant	Symbol	Value
Bohr radius	a_0	$5.29177210544 \times 10^{-11}$ m
Hartree energy	E_h	$4.3597447222060 \times 10^{-18}$ J
Electron mass	m_e	$9.1093837139 \times 10^{-31}$ kg
Proton mass	m_p	$1.67262192595 \times 10^{-27}$ kg
Elementary charge	e	$1.602176634 \times 10^{-19}$ C (exact)
Planck constant	h	$6.62607015 \times 10^{-34}$ J·s (exact)
Reduced Planck constant	$\hbar$	$1.054571817 \times 10^{-34}$ J·s (exact)
Speed of light	c	299 792 458 m/s (exact)
Boltzmann constant	k_B	1.380649×10^{-23} J/K (exact)
Avogadro constant	N_A	$6.02214076 \times 10^{23}$ mol^{-1} (exact)
Vacuum permittivity	ε_0	$8.8541878188 \times 10^{-12}$ F/m
Bohr magneton	μ_B	$9.2740100657 \times 10^{-24}$ J/T
Nuclear magneton	μ_N	$5.0507837393 \times 10^{-27}$ J/T

Source: NIST CODATA 2022, https://physics.nist.gov/cuu/Constants/ (published November 2024). The four exact constants (e, h, k_B, N_A) have been fixed since the 2019 SI redefinition.

8.6 Python port (g16_au_convert / g16_au_table / g16_gaussian_field_convert)

Unlike MOSurface/RamanBrowser (§9.6, §10.2), au2SI's Python port is folded directly into the G16parser package itself, not kept as a separate companion: it has no dependency of its own beyond plain numpy, and is exactly the kind of general, version-agnostic utility (like g16_read_input) that belongs in the core Python package rather than alongside it.

```
import G16parser as g16

g16.g16_au_convert(-875.932, 'energy', 'au2si', target='kJ/mol')
g16.g16_au_convert(6.3347, 'dipole', 'si2au', target='Debye')
g16.g16_au_table('energy')
g16.g16_gaussian_field_convert(10, 'g2phys', unit='V/Ang')
```

Function	Signature
g16_au_convert	g16_au_convert(value, quantity, direction, target='', verbose=None) – same quantity strings as AU_convert (§8.1); quantity='help' prints the reference table and returns None
g16_au_table	g16_au_table(section='') – same section filter as AU_table (§8.2), plus 'constants'
g16_gaussian_field_convert	g16_gaussian_field_convert(value, direction, unit='au', verbose=True)

Validation. Cross-checked directly against the MATLAB original, not just internally self-consistent: 12 quantity/target combinations spanning every category (energy, length, dipole, polar, beta, force, efield, temp, mass, pressure, frequency) agree with AU_convert.m to ∼1e-13–1e-14 relative difference (limited by %.12e text-transfer precision, not a real discrepancy); both G_gaussian_field_convert.m directions (g2phys/ phys2g) match exactly. A 23-case pytest regression suite (tests/test_au_convert.py) covers every function, including array input, the 'help' mode, error paths, and the rounding-warning path on g16_gaussian_field_convert.

9 MOSurface: real-space orbital, density, and ESP isosurfaces

`MOSurface/` provides four GaussView-equivalent real-space renderers, all built directly from the raw `.fchk` basis-set data (or, for the fourth, directly from a saved `.cube` file) with no toolbox or third-party dependency beyond MATLAB's own built-in `isosurface`/`isonormals`/`patch`/`contourf`: `G_draw_mo_surface` (§9.1, a chosen molecular orbital's ±isosurface or 2D nodal map, GaussView's "Molecular Orbitals"), `G_draw_density_surface` (§9.2, the total electron density, a density *difference* between two calculations, or the *spin* density of an unrestricted calculation, GaussView's "Total Density"/"Spin Density"), `G_draw_esp_surface` (§9.3, the electrostatic potential mapped onto the density envelope, GaussView's "ESP mapped on electron density"), and `G_draw_cube_surface` (§9.4, which re-renders an isosurface straight from an already-saved `.cube` file – no `.fchk`, no basis-set re-evaluation). The first three can each export the underlying volumetric grid to a Gaussian-format `.cube` file via `'SaveCube'` (see each subsection); `G_draw_cube_surface` is the load-and-replot companion that reads one back (or a real `cubegen`/GaussView-generated `.cube` just as well). Every other `MOSurface/` file (`g_`-prefixed, lowercase) is an internal helper, never called directly by the user — only the four `G_`-prefixed functions above are the public API. A Python port of all four, `MOSurface/python/`, also exists (see §9.6).

9.1 G_draw_mo_surface

```
h = G_draw_mo_surface(data, mo_index)
h = G_draw_mo_surface(data, mo_index, 'Name', Value, ...)
```

`data` is the full struct returned by `G09_fchk_read`/ `G16_fchk_read` (must contain `.filename`, `.mol`, `.Nbasis`, `.alpha_MO_coeff`, `.xyz_bohr`). `mo_index` selects which orbital to render: either a positive integer (1-based column of `reshape(data.alpha_MO_coeff, data.Nbasis, data.Nbasis)`), or the string `'HOMO'`, `'LUMO'`, `'HOMO-n'`, `'LUMO+n'` (resolved against `data.HOMO_idx`).

Option	Default	Description
'Mode'	'surface'	'surface': 3D ±isosurface (below). 'contour': a 2D filled-contour amplitude map on a single plane, with the nodal line drawn explicitly – see §9.1
'Plane'	'auto'	('contour' only) 'auto': best-fit plane through all atoms (PCA). 'xy'/'xz'/'yz': a coordinate plane through the centroid
'PlaneOffset'	0 Å	('contour' only) shifts the evaluation plane along its own normal – see §9.1
'IsoValue'	0.02	('surface' only) Wavefunction-amplitude isosurface level (a.u.), GaussView's own default. Both +IsoValue and −IsoValue lobes are drawn.
'GridSpacing'	0.15 Bohr	Real-space grid spacing
'Padding'	4.0 Bohr	Grid/plane padding around the molecule's bounding box; auto-expanded if needed in 'surface' mode (see below)
'SaveCube'	'' (off)	('surface' only) filename; if given, writes the signed MO-amplitude grid already computed for the isosurface (no extra evaluation) to a Gaussian-format MO .cube file (negative atom count plus the orbital-index line, matching real cubegen MO cubes)
'PosColor'	blue	Positive-amplitude colour
'NegColor'	red	Negative-amplitude colour
'FaceAlpha'	0.55	('surface' only) Lobe transparency; used as-is with 'SurfaceStyle','transparent' (the default), overridden to 1.0 with 'solid' unless given explicitly (which always wins), unused with 'grid'
'SurfaceStyle'	'transparent'	('surface' only) 'solid' | 'transparent' | 'grid' – as in GaussView. 'grid' draws the isosurface as a wireframe (mesh edges only, no face fill) coloured by PosColor/NegColor instead of a filled patch. Neither this function nor G_draw_density_surface decimates the 'surface' mesh, so a fine 'GridSpacing' can give a very dense wireframe – coarsen it for a readable grid render
'ShowMolecule'	true	'surface': overlay the CPK model via whichever of G09_draw_molecule/G16_draw_molecule is on the path. 'contour': overlay a lightweight 2D atom/bond sketch instead
'ShowLabels'	false	('surface' only) Atom labels, forwarded to the molecule drawer; always shown in 'contour' mode
'Ax'	[] (new figure)	Existing axes handle
'Verbose'	true	Print grid size and timing

Returns a struct h. In 'surface' mode: h.pos/h.neg, the positive/negative lobe patch handles ([] if that lobe was not found at IsoValue). In 'contour' mode: h.contour (filled-contour handle) and h.zero (nodal-line handle, [] if the amplitude never changes sign on the plane).

The physics

Each contracted Gaussian-type basis function is evaluated directly on the grid from its primitives,

$$N(\alpha, l, m, n) = \left(\frac{2\alpha}{\pi}\right)^{3/4} \sqrt{\frac{(4\alpha)^{\,l+m+n}}{(2l-1)!!\,(2m-1)!!\,(2n-1)!!}},$$

the standard Cartesian Gaussian-type-orbital (GTO) normalization constant [1]. **Pure D/F/G shells** are built from the individually-normalized Cartesian components combined into real solid harmonics using the polynomial coefficients of Ribaldone & Desmarais [5] (a modern, explicit re-derivation of the classic Schlegel & Frisch transformation [4]); the overall normalization of each pure component is *computed at runtime*, not hard-coded, from the exact combinatorial self-overlap of same-centre Cartesian Gaussians — deliberately avoiding hand-derived constants for the higher (F, G) shells, after an arithmetic slip was caught while hand-checking the simpler D case during development. **Cartesian D/F/G** shells use the corresponding per-component normalization ratio, computed the same way. **Shell-component ordering** (including the native Gaussian `.fchk` reversal for Cartesian shells with $L \geq 4$) follows the convention documented by the IOData project's Gaussian `.fchk` reader [6]. All of this is summed directly into a single `[Ngrid x 1]` accumulator, shell by shell, never forming the full `[Ngrid x Nbasis]` basis matrix, so memory stays bounded regardless of basis-set size.

Supported shells: S, P, SP, D, F, G (pure or Cartesian). A basis set containing H (or higher) shells raises a clear, immediate error rather than silently omitting or mis-rendering those basis functions — there is no partial/best-effort rendering mode. F/G support was validated on a real Au-containing basis set (pure F and pure G functions on the Au atom): every molecular orbital with significant weight on those shells gave a self-normalization integral $\int |\psi|^2\, dV$ of 0.998–1.000.

Deep-core and some high-virtual orbitals can show grid artifacts. A uniform rectangular grid under-resolves very tight (high-exponent) primitives: for such orbitals, the self-normalization integral can be non-monotonic and fail to converge cleanly even as `'GridSpacing'` is refined (root cause: a narrow, sharply-peaked Gaussian is intrinsically hard for naive Riemann-sum quadrature to capture, not a bug in the evaluator). Verified on real data: 10 of 12 ordinary valence/frontier orbitals sampled across a full basis set gave 0.993–1.000; the deepest core orbital and the highest virtual (both dominated by very tight primitives) did not converge under grid refinement. This does not affect the chemically relevant use case of rendering valence/frontier (HOMO/LUMO-region) orbitals.

Automatic padding. Some orbitals (typically diffuse high virtuals) extend well beyond the default 4 Bohr padding and would otherwise be visibly clipped — a flat-cut isosurface face at the edge of the plotted box. After each isosurface extraction, the vertices are checked against the grid boundary; if any lie within 1.5 grid cells of it, `'Padding'` is multiplied by 1.6 and the whole grid/evaluation/extraction is redone, up to 3 times (subject to the same 30-million-point grid-size cap that also applies to a manually-set `'Padding'`). If an explicit `'Padding'` was given, this auto-expansion is skipped and a warning recommends increasing it instead.

```
data = G16_fchk_read('4-NTP.fchk');
G_draw_mo_surface(data, 'HOMO');
G_draw_mo_surface(data, 'LUMO+1', 'IsoValue', 0.03, 'GridSpacing',
   0.12);
G_draw_mo_surface(data, 'HOMO', 'SaveCube', 'homo.cube');   % also
```

```
   export
G_draw_mo_surface(data, 'HOMO', 'SurfaceStyle', 'grid', 'GridSpacing',
    0.3);
```

'Mode','contour': 2D nodal maps

Instead of a 3D isosurface, `'Mode','contour'` evaluates the MO on a single 2D plane and renders a filled amplitude map (diverging colour scale built from `'NegColor'`/`'PosColor'`, pinned to white at zero regardless of the actual data range) with the nodal (zero-amplitude) line drawn explicitly in black, plus a lightweight 2D atom/bond sketch for reference. It is much cheaper than a 3D grid, and often clearer for reading off the nodal pattern of a planar/near-planar π system than a 3D isosurface viewed from an angle – compare the worked example below with the equivalent `'Mode','surface'` render of the same orbital.

> **A π-symmetry orbital is exactly zero on its own molecular plane.** By symmetry, a π MO's amplitude is antisymmetric about the plane it is built perpendicular to, so `'Plane','auto'` (the best-fit plane through the atoms) together with the default `'PlaneOffset',0` shows essentially nothing for a typical aromatic HOMO/LUMO (verified: max amplitude $\sim 10^{-9}$–10^{-29} a.u., pure numerical noise, on two different real aromatic test molecules). This is a property of the orbital, not a bug. `'PlaneOffset'` shifts the evaluation plane along its own normal by the given number of Å (typically 0.5–1.0 for a π system), probing a plane parallel to the molecular plane where the π density actually peaks; the atom sketch is always drawn at the true, unshifted positions, for reference. A warning is printed whenever the resulting amplitude is essentially zero everywhere, naming this as the likely cause.

```
% A HOMO with pi character: offset=0 shows (near) nothing by symmetry
G_draw_mo_surface(data, 'HOMO', 'Mode', 'contour');

% The pi density itself, 0.7 Angstrom above the ring
G_draw_mo_surface(data, 'HOMO', 'Mode', 'contour', 'PlaneOffset', 0.7)
   ;
```

See also `G09_fchk_read`/`G16_fchk_read`, `G09_draw_molecule`/`G16_draw_molecule` (main toolbox manual).

9.2 G_draw_density_surface

```
h = G_draw_density_surface(data)
h = G_draw_density_surface(data, 'Name', Value, ...)
h = G_draw_density_surface(data, 'CompareTo', data2, ...)
h = G_draw_density_surface(data, 'SpinDensity', true, ...)
```

Renders the total electron density, or (given a second structure via `'CompareTo'`) a density *difference* map, or (for an unrestricted calculation, via `'SpinDensity'`) the *spin* density $\rho_\alpha - \rho_\beta$ – a real-space equivalent of GaussView's "Total Density"/"Spin Density" surface types. It shares its real-space evaluation engine with `G_draw_mo_surface` (§9.1): the density is electron-count-exact, not a heuristic, built from

$$\rho(\mathbf{r}) = \sum_i n_i \, |\psi_i(\mathbf{r})|^2$$

summed over all *occupied* orbitals of the relevant spin channel ($n_i = 2$ per orbital for the total/difference density, closed shell; $n_i = 1$ per orbital, alpha and beta separately, for `'SpinDensity'`), evaluated in a *single* pass through the basis shells for every occupied orbital at once (`g_eval_mo_on_grid`

accepts an `[Nbasis x K]` coefficient matrix, not just a single column, and returns an `[Ngrid x K]` result) rather than looping the whole per-shell evaluation once per orbital.

Most options (`'Mode'`, `'Plane'`, `'PlaneOffset'`, `'GridSpacing'`, `'Padding'` with the same auto-expansion, `'FaceAlpha'`, `'SurfaceStyle'`, `'ShowMolecule'`, `'ShowLabels'`, `'AtomScale'`, `'Ax'`, `'Verbose'`) are identical to `G_draw_mo_surface`. The differences:

Option	Description
`'CompareTo'`	A second data struct (same requirements as `data`). If given, renders ρ(CompareTo) − ρ(data) instead of the total density of `data` alone – e.g. field-on minus field-off, to see where charge density increases/decreases. Both densities are evaluated on the same grid/plane, built from `data`'s geometry. Mutually exclusive with `'SpinDensity'` (default: `[]`, total density only)
`'SpinDensity'`	Logical; if `true`, renders $\rho_\alpha(\mathbf{r})-\rho_\beta(\mathbf{r})$ of `data` alone – see the tbnote below for the open-shell requirement. Mutually exclusive with `'CompareTo'` (default: `false`)
`'IsoValue'`	Default `0.001` e/Bohr^3 – the classic isodensity value conventionally used to define a molecule's outer envelope/"size", appropriate for the *total* density. A difference or spin density is usually one or more orders of magnitude smaller; if no isosurface is found, a warning suggests reducing `'IsoValue'`
`'SaveCube'`	(`'surface'` only) filename; if given, writes the density (total, *difference* with `'CompareTo'`, or *spin* with `'SpinDensity'`) grid already computed for the isosurface to a Gaussian-format `.cube` file, at no extra evaluation cost (default: `''`, off)
`'PosColor'`	The (only) density lobe colour without `'CompareTo'`/`'SpinDensity'`; the positive-difference or alpha-excess lobe colour with either
`'NegColor'`	Only used with `'CompareTo'` (negative-difference lobe) or `'SpinDensity'` (beta-excess lobe)

Returns `h.pos`/`h.neg`/`h.contour`/`h.zero`, with the same meaning as `G_draw_mo_surface`; without `'CompareTo'`/`'SpinDensity'`, `h.neg` and `h.zero` are always `[]` (a total density is single-signed, so there is no negative lobe and no zero/nodal line to trace).

Closed-shell only for the total density/`'CompareTo'`; a dedicated open-shell path for `'SpinDensity'`. The total-density and `'CompareTo'` paths require `data.Nalpha == data.Nbeta` (and the same for `'CompareTo'`, if given) and raise a clear error otherwise, rather than silently building a wrong electron count for an open-shell (UHF/UKS) calculation. `'SpinDensity'` is the dedicated open-shell path instead: it does *not* gate on `Nalpha==Nbeta` (not itself a restricted/unrestricted signal – a broken-symmetry biradical singlet, for instance, can have `Nalpha==Nbeta` with genuinely different alpha/beta orbitals), but instead re-reads the raw `"Beta MO coefficients"` `.fchk` section directly (in the same self-contained spirit as `g_read_aobasis_from_fchk`, no core-toolbox change), which Gaussian only ever writes for a genuinely unrestricted calculation – a restricted (RHF/RKS) `.fchk` has identical alpha/beta orbitals by construction (zero spin density everywhere) and this section is simply absent, raising a clear error rather than a confusing empty or all-zero plot. Each spin channel then uses occupation 1 per orbital (not 2), via `g_eval_density_on_grid`'s optional `occ_factor` argument.
Validated first on synthetic test cases (no genuine UHF/UKS `.fchk` was available in this repository at initial development time): a beta channel copied byte-for-byte from the alpha

channel gives *exactly* zero spin density at every sampled point (both a direct pointwise check and the expected "no isosurface found" warning); a beta channel using a shifted occupied window (still `Nbeta` orthonormal columns of the same AO basis, but a genuinely different occupied subspace) gives a clearly nonzero, spatially-varying spin density at 78% of sampled points.
Subsequently validated end-to-end against a real Gaussian UKS calculation: the methyl radical ($CH_3^\bullet$, UB3LYP/6-31G(d), doublet, $N_\alpha = 5$, $N_\beta = 4$). Integrating the rendered spin density over the same real-space grid gives $\int(\rho_\alpha - \rho_\beta)\,dV = 1.0000$, matching $N_\alpha - N_\beta = 1$ (the one unpaired electron) essentially exactly – a quantitative check, not just a qualitative one. The isosurface shape matches the textbook picture for this system's SOMO (a carbon p_z orbital, perpendicular to the molecular plane): the positive (alpha-excess) lobe spans $z \in [-1.14, +1.19]$ Å – i.e. two separate lobes straddling the (planar) molecule, not a single blob centred on it – while a smaller negative (beta-excess) lobe sits close to the C-H bonding region (xy-radius from the carbon up to 1.44 Å), consistent with the well-known spin polarization at the hydrogens of $CH_3^\bullet$ (the same effect responsible for their negative ^{1}H hyperfine coupling constant in EPR).

A raw density plot on a plane through the nuclei is dominated by sharp nuclear cusps. The electron density peaks very sharply at each nucleus – one to several orders of magnitude larger than the chemically-interesting bonding/valence-region values – so a `'Mode','contour'` colour scale naively set to the data's true maximum saturates the whole map to near-white except at a few nuclear pinpoints, hiding the region of actual interest. `'Mode','contour'` therefore caps the colour scale at the 98th percentile of the plotted values instead (values above the cap simply saturate to the top-of-scale colour); `'Mode','surface'` is unaffected, since an isosurface at a single, comparatively low `'IsoValue'` (e.g. the default 0.001) sits well outside the nuclear cusp region already.

```
data = G16_fchk_read('4-NTP.fchk');
G_draw_density_surface(data);                    % total density,
   0.001 envelope
G_draw_density_surface(data, 'Mode', 'contour');  % 2D map, capped
   colour scale

data1 = G16_fchk_read('nofield.fchk');
data2 = G16_fchk_read('field_x025.fchk');
G_draw_density_surface(data1, 'CompareTo', data2, 'IsoValue', 0.0005);
G_draw_density_surface(data, 'SaveCube', 'density.cube');   % also
   export
G_draw_density_surface(data, 'SurfaceStyle', 'grid', 'GridSpacing',
   0.3);

radical = G16_fchk_read('radical_uks.fchk');   % unrestricted (UHF/UKS
   )
G_draw_density_surface(radical, 'SpinDensity', true, 'IsoValue',
   0.002);
```

See also `G_draw_mo_surface` (§9.1), `G09_fchk_read`/`G16_fchk_read`.

9.3 G_draw_esp_surface

```
h = G_draw_esp_surface(data)
h = G_draw_esp_surface(data, 'Name', Value, ...)
h = G_draw_esp_surface(data, 'CompareTo', data2, ...)
```

Renders the molecular electrostatic potential (ESP) mapped onto the total-density isosurface (built via `G_draw_density_surface`'s machinery) – a real-space equivalent of GaussView's classic "ESP mapped on electron density" surface, showing electrophilic (`PosColor`, blue, $V > 0$) vs. nucleophilic (`NegColor`, red, $V < 0$) character:

$$V(\mathbf{r}) = \sum_A \frac{Z_A}{|\mathbf{r} - \mathbf{R}_A|} - \int \frac{\rho(\mathbf{r}')}{|\mathbf{r} - \mathbf{r}'|}\, d\mathbf{r}'$$

Z_A is read from the `.fchk "Nuclear charges"` section, *not* the atomic number: for an ECP (effective core potential) atom this is the reduced effective charge seen by the explicit electrons – verified on a real Au-ECP test file, `"Nuclear charges"` = 19 for Au vs. atomic number 79 (a 60-electron core ECP); using the atomic number would silently give a badly wrong nuclear term for any such atom.

Unlike `G_draw_mo_surface`/`G_draw_density_surface`, the electronic term is a genuine Coulomb integral, not a simple point evaluation. By default it is evaluated *exactly* (up to double-precision/basis-set truncation), via the analytic McMurchie-Davidson Hermite-Gaussian recursion [3, 2] – treating each surface vertex as an arbitrary field point in the same integral family as the nuclear-attraction term, built from Hermite expansion coefficients E_t^{ij}, Hermite Coulomb integrals R_{tuv}, and the Boys function $F_n(x)$ (evaluated in closed form via the regularized incomplete gamma function). This only supports S, P, SP, and pure D shells; for a basis with Cartesian D or F/G shells (or if `'ESPMethod'` is explicitly `'numeric'`), it falls back to the older *numerical* Coulomb-grid-sum method (see the tbnote below), with a printed notice if `'Verbose'`. The full derivation, and the validation methodology and results summarized below, are in `Theory_Vibrations_Polarizability.tex`, Part III.

Option	Default	Description
'IsoValue'	0.001	Density isovalue defining the surface shape (as in G_draw_density_surface)
'GridSpacing' / 'Padding'	0.15 / 4.0 Bohr	Surface-*shape* grid (auto-expanding padding, as in G_draw_mo_surface)
'MaxVertices'	3000	The density mesh is decimated (MATLAB's built-in reducepatch) to at most this many vertices before the ESP evaluation, since ESP is smooth and a full-resolution mesh (often >10000 vertices) is unnecessary and would make it far slower than needed
'CompareTo'	[]	A second data struct (same requirements as data). If given, renders V(CompareTo) − V(data) instead of the absolute ESP of data alone – e.g. field-on minus field-off. Both ESPs are evaluated independently (own analytic/numerical fallback, nuclear charges, density matrix) at the SAME vertices, built from data's density-surface shape only, so the comparison is physically meaningful only if the two structures share the same atomic coordinate frame
'ESPMethod'	'auto'	'auto': analytic when the basis is within scope, else numerical fallback (with notice). 'analytic': force analytic, erroring on an out-of-scope basis. 'numeric': force the numerical method regardless of basis
'ESPGridSpacing'	0.20 Bohr	*Only used by the numerical method* (auto fallback or forced): spacing of the separate density grid used for the Coulomb sum – see the accuracy note below
'ESPPadding'	6.0 Bohr	*Numerical method only*: padding for that grid; empirically far less important than 'ESPGridSpacing'
'DistanceFloor'	0.5×'ESPGridSpacing'	*Numerical method only*: minimum distance used in the Coulomb sum, avoiding a spurious spike when a grid point lands very close to a vertex
'SaveCube'	'' (off)	Filename; if given, *additionally* evaluates ESP on a separate, dedicated volumetric grid (see 'CubeSpacing'/'CubePadding' – NOT the surface-shape grid, NOT just the decimated vertices) and writes it to a Gaussian-format .cube file. This is a genuinely extra computation: ESP is otherwise only ever evaluated at the (few-thousand-at-most) decimated vertices
'CubeSpacing'	0.30 Bohr	Grid spacing for 'SaveCube', coarser by default than 'GridSpacing'/'ESPGridSpacing' – with the analytic method, cost scales roughly with N_{basis}^2 per point, so a fine cube over many points can be genuinely slow (a 2-million-point safety cap raises a clear error rather than starting an impractically long run)
'CubePadding'	4.0 Bohr	Padding for 'SaveCube'
'PosColor' / 'NegColor'	blue / red	Standard ESP convention (electrophilic / nucleophilic); with 'CompareTo', positive/negative *difference* instead
'FaceAlpha'	1.0	Used as-is with 'SurfaceStyle','solid' (the default here, unlike G_draw_mo_surface/G_draw_density_surface's 'transparent' default – an ESP map is conventionally an opaque painted surface); overridden to 0.55 with 'transparent' unless given explicitly;

Returns `h.surf`, the coloured surface patch handle.

> **The analytic method is validated end-to-end against real Gaussian ESP data; the numerical fallback is not publication-grade.** A head-to-head comparison against a real Gaussian-generated ESP `.cube` file (`cubegen`, Full Density Matrix option), sampled at the *actual* density-isosurface distance range used by this function (2.4–4.4 Bohr from the nearest nucleus, the default `IsoValue=0.001`): the default analytic method gave correlation 1.0000000 and an RMS error of $\sim 0\%$ of the true ~ 0.14 Hartree/e signal span; the numerical fallback, at its best-tested setting, gave correlation 0.03 (essentially none) and an RMS error 54% of that span. The numerical method's own internal aliasing issue that drives this: a uniform grid systematically mis-estimates the raw density right at each nuclear cusp (the same phenomenon documented for individual MOs in §9.1) – verified separately, the total electron count captured by a naive grid sum ranged from -2% to $+31\%$ depending on `'ESPGridSpacing'` alone, *non*-monotonically. The density grid is rescaled to the exact electron count (`data.Nelec`) before the Coulomb sum, which removes the dominant, roughly-uniform component of this bias but not its remaining atom-to-atom spatial variation – which is why, even with this correction, the numerical method's correlation against real Gaussian ESP data is essentially zero at realistic surface distances. **Use the numerical fallback (`'ESPMethod','numeric'`, or the automatic fallback for Cartesian-D/F/G bases) for a qualitative electrophilic/nucleophilic pattern only, never for quantitative ESP values.**

```
data = G16_fchk_read('4-NTP.fchk');
G_draw_esp_surface(data);                          % exact, analytic (
    default)
G_draw_esp_surface(data, 'MaxVertices', 1500);    % faster, same
    accuracy

data2 = G16_fchk_read('field_on.fchk');
G_draw_esp_surface(data, 'CompareTo', data2);     % ESP difference map

G_draw_esp_surface(data, 'SaveCube', 'esp.cube', 'CubeSpacing', 0.5);
G_draw_esp_surface(data, 'SurfaceStyle', 'grid', 'MaxVertices', 800);
```

See also `G_draw_density_surface` (§9.2), `G_draw_mo_surface` (§9.1), `G09_fchk_read`/`G16_fchk_read`.

9.4 G_draw_cube_surface

```
h = G_draw_cube_surface(cubefile)
h = G_draw_cube_surface(cubefile, 'Name', Value, ...)
h = G_draw_cube_surface(cubefile, 'ColorBy', esp_cubefile, ...)
```

Renders an isosurface straight from a saved Gaussian-format `.cube` file – the load-and-replot companion to the other three functions' own `'SaveCube'` option, and equally happy with a real `cubegen`/ GaussView-generated `.cube`. No `.fchk` is read and no basis-set data is re-evaluated: the scalar field and the atoms are both read directly from `cubefile` by `g_read_cube_file`, and MATLAB's own `isosurface` is applied to it exactly as the other three functions apply it to a freshly-evaluated grid.

Option	Default	Description
'ColorBy'	''	Path to a SECOND .cube file. If given, the isosurface SHAPE still comes from cubefile (a single positive-value envelope, as for a density cube), but each vertex is coloured by trilinearly interpolating the SECOND cube's field there – reproducing GaussView's classic "ESP mapped on electron density" picture from two independently-saved cube files (a density cube as cubefile, an ESP cube as 'ColorBy'), with no .fchk needed at all. The two cubes must share an IDENTICAL grid (same origin, spacing, and point counts); a clear error is raised otherwise, rather than silently resampling – see the tbnote below
'IsoValue'	guessed	Guessed from cubefile's title text: 0.001 for a density-like field (title contains "density"), 0.02 for an MO-amplitude cube (negative-Natoms header, GaussView's own MO default) – matching G_draw_density_surface/G_draw_mo_surface's own defaults for a cube saved by this toolbox. For anything else (an ESP cube, or an unrecognized/external cube's title) there is no universally sensible default – 'IsoValue' must then be given explicitly, or a clear error explains why
'SurfaceStyle'	'transparent'	'solid' | 'transparent' | 'grid' – as in the other three functions. With 'ColorBy', 'grid''s edges are coloured by interpolating the second cube's field (EdgeColor,'interp'), like G_draw_esp_surface; without it, by 'PosColor'/'NegColor' like the others
'FaceAlpha'	0.55	As in G_draw_density_surface/G_draw_mo_surface: used as-is with 'transparent', overridden to 1.0 with 'solid' unless given explicitly, unused with 'grid'
'PosColor' / 'NegColor'	blue / red	Lobe colours (no 'ColorBy'), or the diverging-colormap endpoints (with 'ColorBy')
'MaxVertices'	5000	The isosurface mesh is decimated (reducepatch) to at most this many vertices, mainly to bound 'ColorBy' interpolation cost
'ShowMolecule'	true	Overlays a CPK model built directly from the atoms stored IN the cube file – no .fchk needed
'ShowLabels' / 'AtomScale' / 'Ax' / 'Verbose'	as elsewhere	Same meaning as in G_draw_mo_surface

Returns h.pos/h.neg (positive/negative lobe patch handles; h.neg is [] for an unsigned field, or whenever 'ColorBy' is given, since that mode always renders a single shape-defining lobe).

Signed-ness and the 'ColorBy' grid requirement. A field is treated as SIGNED (both a +IsoValue and a −IsoValue lobe, coloured 'PosColor'/'NegColor') if it is an

MO cube, or if its values genuinely span both signs beyond numerical noise (an ESP cube loaded alone, for instance, IS signed by this rule); otherwise only the single +`IsoValue` lobe is drawn. `'ColorBy'` overrides this entirely – the shape is always the single envelope of `cubefile`. Because `G_draw_density_surface`/ `G_draw_mo_surface` auto-expand `'Padding'` when the isosurface would otherwise be clipped (§9.1), two cubes saved with the same *nominal* `'GridSpacing'`/ `'Padding'` can still end up on different grids if only one of them triggered that auto-expansion – verified directly: this is exactly what happened on a first attempt at combining a density cube with an ESP cube. Pass an *explicit* `'Padding'` (which disables auto-expansion) matching the ESP cube's `'CubePadding'` exactly to guarantee identical grids.

`'ColorBy'`'s colour scale matches `G_draw_esp_surface`'s own convention, not a naive max. The colour axis is set from the 99th percentile of |colour field| (`g_pctile_local`, the same helper `G_draw_esp_surface` itself uses), *not* the raw maximum – verified to matter directly: an earlier version used `max(abs(...))`, and a single outlier vertex was enough to wash out the entire colour scale compared to `G_draw_esp_surface`'s own rendering of the same data. With the percentile-based fix, the two functions' colour axis limits on the same molecule agree to within ∼0.3% (the small remainder being ordinary trilinear-interpolation error from reading the ESP field off a separately-saved cube grid, rather than evaluating it exactly at each surface vertex the way `G_draw_esp_surface` does internally).

```
% Re-plot a density cube saved earlier -- no .fchk needed:
G_draw_cube_surface('density.cube');

% Re-plot a saved MO cube (sign auto-detected):
G_draw_cube_surface('homo.cube', 'SurfaceStyle', 'grid');

% Classic "ESP mapped on density" from two independently saved cubes,
% entirely without re-reading the .fchk:
G_draw_cube_surface('density.cube', 'ColorBy', 'esp.cube', 'IsoValue',
    0.001);
```

See also `G_draw_density_surface` (§9.2), `G_draw_mo_surface` (§9.1), `G_draw_esp_surface` (§9.3).

9.5 G_relight

```
G_relight                  % current axes, current view
G_relight(ax)              % given axes (or an array of axes)
G_relight(ax, [az el])     % set the view first, then relight
G_relight([], [0 90])      % current axes, view from the top
```

Re-positions the lights of a 3-D axes so that they follow the current (or a new) viewing direction, for figures made by `G_draw_mo_surface`, `G_draw_density_surface`, `G_draw_esp_surface` or `G_draw_cube_surface`. Changing `view` after drawing leaves the lights where they were, and the surfaces turn dark or flat; `G_relight` restores the lighting scheme of the drawing functions: a headlight (light at the camera) plus a second light at azimuth 45° and elevation 30° relative to the camera. Existing lights (the first two created in the axes) are re-used, so repeated calls do not accumulate lights; if the axes has none, they are created with Gouraud lighting.

```
mo = G16_fchk_read('NMA_wB97XD_augTZ_TD_trans.fchk');
G_draw_mo_surface(mo, 'HOMO');
G_relight(gca, [-60 40]);
```

```
exportgraphics(gca, 'HOMO.pdf', 'ContentType', 'vector');
```

9.6 Python port (`MOSurface/python/`)

All four functions above are also ported to Python, as a standalone `mosurface` package in `MOSurface/python/` – deliberately kept as a separate companion rather than folded into the `G16parser` package itself, so that this real-space isosurface engine (with its own heavier dependencies, `scipy` and `scikit-image`, on top of `G16parser`'s own `numpy`/`matplotlib`) does not become a hard dependency of the core Python toolbox. Same naming philosophy as `G16parser`: public functions are prefixed `g16_` (`g16_draw_mo_surface`, `g16_draw_density_surface`, `g16_draw_esp_surface`, `g16_draw_cube_surface`), every other (leading-underscore) name is an internal helper. Options are the same as their MATLAB counterparts, in `snake_case` (`'IsoValue'` → `isovalue`, `'ShowMolecule'` → `show_molecule`, ...). `data` is the `Struct` returned by `G16parser`'s own `g16_fchk_read` – the one real dependency on `G16parser` (for the input struct's shape and for the `g16_draw_molecule` CPK overlay), mirroring `MOSurface/`'s own dependency on `G09_fchk_read`/`G16_fchk_read` and `G09_draw_molecule`/`G16_draw_molecule`.

Validation. The numerically critical parts were cross-checked directly against the MATLAB original on a real `.fchk` (4-NTP, 218 basis functions), not just internally self-consistent: `eval_mo_on_grid`/`eval_density_on_grid` (every S/P/SP/ D/F/G shell, pure and Cartesian) agree with `g_eval_mo_on_grid.m`/`g_eval_density_on_grid.m` to ∼1e-11 relative difference (HOMO/LUMO/HOMO-2/the highest-numbered MO, and the total density); the analytic McMurchie-Davidson ESP evaluator agrees with `g_eval_esp_analytic.m` to ∼4e-13 relative difference; pure D/F/G self-normalization (synthetic single-shell self-overlap, fine-grid Riemann sum) converges to 1.0000 for every component of every supported shell; `.cube` read/write round-trips exactly (to `%.5E` text precision) against a brute-force nested-loop reference. All four functions were then run end-to-end on real `.fchk` files (closed-shell; UKS spin density; a field-on/field-off pair for density/ESP difference), producing chemically sensible pictures – e.g. the expected SOMO lobe perpendicular to the molecular plane for a CH_3 radical's spin density, and the expected heteroatom-localised electron-rich region in ESP-on-density colouring.

Rendering differences from MATLAB (intentional, not a fidelity concern). Isosurface extraction uses `skimage.measure.marching_cubes` in place of `isosurface`/`isonormals` (already returns per-vertex normals from the volume gradient). Mesh decimation (`max_vertices`) uses a simple vertex-clustering simplification instead of `reducepatch` – a different, simpler algorithm, fine for a smooth ESP/`color_by` field. matplotlib's `Poly3DCollection` has no true per-vertex Gouraud shading like `PATCH`: flat-coloured lobes use `shade=True` (simple per-face directional shading), and ESP/ `color_by` surfaces use a per-*face* colour (averaged from its 3 vertices) in place of `FaceColor,'interp'`.

Performance characteristics (read before using on a large system). Pure-Python/NumPy evaluation is markedly slower than MATLAB's compiled engine in two specific ways. (1) The default auto-expanding `padding` re-evaluates the *entire* grid on every retry (up to 4 attempts, ×1.6 each); on a real 218-basis-function molecule, one retry sequence took several minutes where an explicit, sufficient `padding` (`padding_explicit=True`) took ∼4 s – pass an explicit `padding` for routine use rather than relying on the auto-expand default. (2) `g16_draw_esp_surface`'s analytic ESP evaluation has a large *fixed* per-call cost dominated by the number of shell *pairs* (∼11 s fixed

overhead measured for a 104-subshell basis, largely independent of point count) – fine for the normal case (a few thousand *decimated surface vertices*, via `max_vertices`), but this makes `save_cube` with the analytic method over a *full volumetric grid* (potentially hundreds of thousands of points) impractical at realistic resolution, unlike MATLAB, where the same feature is merely slow (per `G_draw_esp_surface.m`'s own "genuinely slow for a large basis" docstring note) rather than impractical. For `save_cube` on `g16_draw_esp_surface`, use a deliberately coarse `cube_spacing`/small `cube_padding`, or `esp_method='numeric'` (faster, less accurate – see G_DRAW_ESP_SURFACE's own "Accuracy" notes above, which apply unchanged here) if a full-resolution ESP cube is genuinely needed. Vectorizing the analytic evaluator's inner shell-pair loop would address this but was out of scope for this initial port, which prioritized exact numerical fidelity over performance.

```
import G16parser as g16
from mosurface import g16_draw_mo_surface, g16_draw_esp_surface

data = g16.g16_fchk_read('4-NTP.fchk')
g16_draw_mo_surface(data, 'HOMO')
g16_draw_esp_surface(data, padding=4.0, padding_explicit=True,
   max_vertices=1500)
```

10 RamanBrowser: interactive Raman/IR stick-spectrum browser

`RamanBrowser/` provides a single click-to-select companion to the core toolbox's own `G16_modeViewer/G09_mode` (text drop-down mode selectors): `G_raman_browser` plots Raman activity (or IR intensity) against frequency as a stick spectrum and lets the mode be picked by clicking directly on its stick – often the more natural way to pick out "that strong peak at ∼1580 cm^{-1}" than first reading its value off a list. Unlike the two `modeViewer`s, which are each tied to one Gaussian major version, `G_raman_browser` is a single function that works with *both*: it auto-detects Gaussian 09 vs. Gaussian 16 (§10.1) and dispatches internally to the matching `G09_/G16_structure/nmodes/draw_mode` functions, unchanged, with no core-toolbox modification. A Python port also exists (§10.2), G16-only.

10.1 G_raman_browser

```
G_raman_browser(filename)
G_raman_browser(filename, 'Name', Value, ...)
```

Opens a window with the requested stick spectrum. Clicking anywhere in the axes selects the mode whose frequency is nearest the click (a red marker highlights it, and an info line above the plot reports its mode number, frequency, symmetry, and intensity). A "Draw mode" button then calls `draw_mode` for the selected mode, opening (or replacing) a 3D structure figure with its displacement arrows.

For `'Property','Raman'` on a file with `CPHF=RdFreq` data (`nm.has_RamanFr`, §10.1), the info line reports *both* the static and the dynamic Raman activity instead of a single value, e.g. `Selected: mode 1, 17.4 cm-1 (A), Raman static = 1.74, 785nm = 1.89` – one `, Nnm = ...` term per incident wavelength beyond the static one, converted from `nm.IncidentLight`'s cm^{-1} values. Without `CPHF=RdFreq` data, or for `'Property','IR'`, the line keeps its original single-value form (e.g. `ir = 4.28`).

Option	Default	Description
`'Property'`	`'Raman'`	`'Raman'` | `'IR'` – which stick intensity to plot. `'Raman'` requires the file to actually contain Raman activities (a `Freq=Raman` job); a clear error suggests `'IR'` instead otherwise
`'Scale'`	`1.5`	Forwarded to `draw_mode`
`'AtomScale'`	`0.35`	Forwarded to `draw_mode`
`'ShowLabels'`	`false`	Forwarded to `draw_mode`

Gaussian 09/16 auto-detection. The Gaussian major version is read from the "Gaussian NN, Revision X.YY," citation line near the top of `filename` via `G16_gaussian_version` (which, despite its name, reads this line for any `NN`); `G09_structure/ G09_nmodes/G09_draw_mode` are used for a detected Gaussian 09 file, `G16_structure/G16_nmodes/ G16_draw_mode` otherwise (including when the version cannot be determined at all, matching this function's original, still most common, use case). This is safe because the two triples' output structs/call signatures are field-for-field and argument-for-argument identical – verified directly on a real Gaussian 09 file (`CH4_NBO.LOG`) and a real Gaussian 16 file (`4-NTP_nbo.out`) side by side, including a simulated stick click on the Gaussian 09 file correctly reaching `G09_draw_mode`.

Static vs. dynamic Raman in the info line. This relies on `G09_nmodes`/`G16_nmodes`'s own `.RamanFr`/ `.IncidentLight`/`.has_RamanFr` fields (§4.7) – `G_raman_browser` adds no parsing of its own, it only formats what those functions already expose. Verified directly on a real 785 nm job (`MPBA_SH.out`): a simulated click on mode 1 produced `Raman static = 1.74, 785nm = 1.89`, matching `nm.Raman(1)` and `nm.RamanFr(1,2)` exactly; the same click on a file with no `CPHF=RdFreq` data produced the plain, unchanged `raman = 0.87` form, with no spurious `Nnm = ...` term.

Window placement on multi-monitor systems. A fixed pixel `Position` for the browser window can land off-screen entirely on a multi-monitor setup whose primary display does not start at virtual desktop coordinate $[0, 0]$ – the window is technically created but never visibly appears (confirmed directly: this happened on first use, and restarting MATLAB was the only workaround until fixed). The same fix already used by `G16_modeViewer` is applied here: a short-lived invisible ordinary `figure` is queried for its default placement to identify the active monitor from `MonitorPositions`, and the browser window is then created directly on that monitor in a single atomic call.

```
% Browse the Raman spectrum, click a peak, press "Draw mode" (G16 file
    ):
G_raman_browser('4-NTP.out');

% Browse IR intensities instead, with a larger displacement-arrow
    scale:
G_raman_browser('4-NTP.out', 'Property', 'IR', 'Scale', 2);

% Works identically on a Gaussian 09 file, auto-detected:
G_raman_browser('indaco.log');
```

See also `G16_modeViewer`, `G16_draw_mode`, `G16_nmodes`, `G16_structure`, `G09_modeViewer`, `G09_draw_mode`, `G09_nmodes`, `G09_structure`, `G16_gaussian_version`.

10.2 Python port (`RamanBrowser/python/`)

`G_raman_browser` is also ported to Python, as a standalone `ramanbrowser` package in `RamanBrowser/python/` – same separate-companion-package status as `MOSurface/python/`'s own `mosurface` package (kept out of `G16parser` itself, see §9.6), and the same naming philosophy: one public function, `g16_raman_browser`, everything else a leading-underscore internal helper. `tkinter` (with an embedded matplotlib `FigureCanvasTkAgg`) replaces `uifigure`/`uiaxes` for the click-to-select stick spectrum window, and `g16_draw_mode`'s own matplotlib window (as already used by `G16parser`'s `g16_mode_viewer`) is opened by the "Draw mode" button, exactly mirroring the MATLAB original's behaviour.

```
from ramanbrowser import g16_raman_browser

g16_raman_browser('4-NTP.out')
g16_raman_browser('4-NTP.out', property='ir', scale=2)
```

G16-only (a real scope difference from the MATLAB original). `G16parser` only targets Gaussian 16 output (a deliberate, documented scope — §9.6's own note on why no `G09parser` exists), so this port cannot dispatch to a Gaussian 09 parser the way `G_raman_browser.m` does via `G09_structure`/`G09_nmodes`/`G09_draw_mode` – there is noth-

ing on the Python side to dispatch *to*. Passing a Gaussian 09 file is unsupported (not silently wrong – `g16_nmodes` still expects Gaussian 16 formatting).

Static vs. dynamic Raman (`.RamanFr`), forward-compatible. The info line's static/dynamic Raman split (§10.1) is read via `getattr(nm, 'has_RamanFr', False)`: since `G16parser`'s own `g16_nmodes` does not yet expose `.RamanFr`/`.IncidentLight`/`.has_RamanFr` (a separate, already-known port gap), this currently always falls back to the plain single-value form – correctly, with no error or spurious output – and will pick up the enhanced display automatically, with no changes needed here, once/if that field is ported to `G16parser`.

Validation. Verified end-to-end with a simulated click (a real `matplotlib.backend_bases.MouseEvent`, not just a direct function call) on `4-NTP_nbo.out`: selecting the same stick produced the identical result as the MATLAB original on the same file — `Selected: mode 10, 528.5 cm-1 (A), raman = 0.25` — and the "Draw mode" button correctly invoked `g16_draw_mode` with mode index 10.

11 Orbital energy analysis

11.1 G09_orbital_energies / G16_orbital_energies

```
oe = G09_orbital_energies(filename)
oe = G09_orbital_energies(filename, 'step', 'last')
oe = G09_orbital_energies(filename, 'step', N)
```

Extracts molecular orbital energies (HOMO/LUMO and the full occupied/virtual spectrum) from a Gaussian 09/16 output file, by parsing the `"Alpha occ. eigenvalues --"` / `"Alpha virt. eigenvalues --"` lines printed by Gaussian's population analysis (and the corresponding `"Beta"` lines for open-shell calculations). These blocks repeat once per SCF calculation in the file (e.g. once per optimisation step); the `'step'` option selects which block to report, consistently with `G09_energy`/`G09_structure`.

Option	Default	Description
`'step'`	`'last'`	Which eigenvalue block to use: `'first'`, `'last'`, or an integer index

Output struct `oe` (energies in Hartree unless noted):

Field	Type	Description
`.alpha_occ`	vector	Occupied alpha orbital energies
`.alpha_virt`	vector	Virtual alpha orbital energies
`.beta_occ`	vector	Occupied beta orbital energies (`[]` if closed-shell)
`.beta_virt`	vector	Virtual beta orbital energies (`[]` if closed-shell)
`.has_beta`	logical	`true` for open-shell (UHF/UKS) calculations
`.HOMO` / `.LUMO`	double	Highest occupied / lowest unoccupied orbital energy
`.gap`	double	`LUMO -- HOMO`
`.HOMO_alpha` / `.LUMO_alpha`	double	Alpha HOMO/LUMO (`==` `.HOMO`/`.LUMO` for closed-shell)
`.HOMO_beta` / `.LUMO_beta`	double	Beta HOMO/LUMO (`[]` if closed-shell)
`.HOMO_eV` / `.LUMO_eV` / `.gap_eV`	double	Same three quantities, in eV
`.step`	int	Block index actually used
`.Nsteps`	int	Number of eigenvalue blocks found in the file
`.filename`	char	Source file name

```
oe = G09_orbital_energies('V_E00t.out');
fprintf('HOMO-LUMO gap = %.3f eV\n', oe.gap_eV);
```

> For open-shell (UHF/UKS) calculations, `HOMO` and `LUMO` are computed as $\max(\mathrm{HOMO}_\alpha, \mathrm{HOMO}_\beta)$ and $\min(\mathrm{LUMO}_\alpha, \mathrm{LUMO}_\beta)$ respectively, so `.gap` always refers to the smallest alpha/beta HOMO–LUMO gap. No differences between the G09 and G16 versions beyond the parsed file's origin.

11.2 G09_draw_orbital / G16_draw_orbital

```
G09_draw_orbital(oe)
G09_draw_orbital(oe, 'Name', Value, ...)
```

Draws a molecular-orbital energy-level diagram from the struct returned by `G09_orbital_energies`/`G16_orbita` showing a user-selected number of occupied/virtual levels around the frontier orbitals and highlighting the HOMO–LUMO transition with an arrow labelled with the gap value.

Option	Default	Description
`'NLevels'`	`5`	Number of occupied/virtual levels shown on each side of the frontier orbitals
`'Units'`	`'eV'`	Energy units: `'eV'` or `'Hartree'`
`'Ax'`	(new figure)	Draw into an existing axes handle
`'OccColor'`	`[0.25 0.25 0.70]`	Colour for non-frontier occupied levels
`'VirtColor'`	`[0.70 0.30 0.25]`	Colour for non-frontier virtual levels
`'HOMOColor'`	`[0.00 0.45 0.85]`	Highlight colour for the HOMO
`'LUMOColor'`	`[0.90 0.35 0.00]`	Highlight colour for the LUMO
`'ArrowColor'`	`[0.15 0.60 0.15]`	HOMO→LUMO transition arrow colour
`'ShowLabels'`	`true`	Annotate each level with its energy value
`'ShowSpins'`	`true`	Draw a paired-electron arrow glyph (↑↓) on occupied levels
`'FontSize'`	`8`	Font size for level energy-value labels
`'FrontierFontSize'`	`13`	Font size for the HOMO/LUMO tags and the gap annotation
`'ArrowHeadSize'`	`0.2`	HOMO–LUMO arrow head size, as a fraction of the arrow length (passed to `quiver`)
`'Title'`	filename or `'Orbital Energy Diagram'`	Figure title

```
oe = G09_orbital_energies('a1.out');
G09_draw_orbital(oe)
G09_draw_orbital(oe, 'NLevels', 8, 'Units', 'Hartree')
```

> Requires the `oe` struct returned by `G09_orbital_energies`/ `G16_orbital_energies` and errors if `alpha_occ`/`alpha_virt` are missing or empty. No differences between the G09 and G16 versions beyond the input struct's origin.

12 Response properties (G16 only)

12.1 G16_hyperpolar

```
hp = G16_hyperpolar(filename)
hp = G16_hyperpolar(filename, 'units', 'au')    % default
hp = G16_hyperpolar(filename, 'units', 'esu')   % 10^-30 esu
hp = G16_hyperpolar(filename, 'units', 'SI')    % 10^-50 C^3 m^3 J^-2
```

Extracts the dipole hyperpolarisability β (static, dynamic, and vibrational contributions) from a Gaussian 16 output file.

Option	Default	Description
'units'	'au'	'au' | 'esu' (10^{-30} esu) | 'SI' (10^{-50} $C^3m^3J^{-2}$)

Field	Description
.beta0.par_z, .beta0.perp_z	Parallel / perpendicular component of static $\beta(0;0,0)$ along z
.beta0.vec_x/y/z, .beta0.beta_vec	Vector components and magnitude $\lvert\beta_{\text{vec}}\rvert$
.beta0.tensor	All Cartesian tensor components $(xxx, xxy, \ldots, zzz)$
.beta_dyn	Struct array, one entry per laser frequency: .lambda_nm, .par_z, .perp_z, .vec_x/y/z, .beta_vec, .tensor
.N_dyn	Number of dynamic (laser) frequencies found
.beta_vib, .has_vib	Diagonal vibrational hyperpolarisability, logical
.filename	

```
hp = G16_hyperpolar('V_E00t.out');
hp.beta0.beta_vec          % static beta modulus, a.u.
hp.beta_dyn(1).lambda_nm   % wavelength of the first dynamic entry (nm
   )
hp.beta_dyn(1).beta_vec    % |beta| at that frequency
```

12.2 G16_tddft

```
td = G16_tddft(filename)
td = G16_tddft(filename, 'nstates', N)      % first N states only
td = G16_tddft(filename, 'plot', true)      % also generate UV-Vis plot
td = G16_tddft(filename, 'FWHM_eV', 0.3)    % broadening FWHM (eV)
```

Extracts TD-DFT excited states and generates a Gaussian-broadened UV-Vis spectrum. For job selection in --Link1-- chains, dominant-excitation labels and orbital-level diagrams see G16_td_plot (§12.3) and G16_td_levels (§12.4).

Option	Default	Description
'nstates'	Inf	Load only the first N excited states
'plot'	false	Generate the UV-Vis plot
'FWHM_eV'	0.30	Gaussian broadening FWHM (eV)

Field	Description
`.n`, `.mult`	State index; multiplicity label (e.g. `'Singlet-A'`, `'Triplet-B'`)
`.eV`, `.nm`, `.f`	Excitation energy (eV), wavelength (nm), oscillator strength
`.S2`, `.has_S2`	$\langle S^2 \rangle$ expectation value (0 for pure singlets), logical
`.trans`	Cell array, one $[N_{\text{contrib}} \times 3]$ matrix per state: columns `[MO_from, MO_to, coeff]` (`MO_to`<0 denotes a de-excitation)
`.Nstates`	Total number of excited states loaded
`.x_nm`, `.eps_cont`	Wavelength grid (100–1000 nm), Gaussian-broadened UV-Vis spectrum (arb. units)
`.FWHM_eV`, `.filename`	

```
td = G16_tddft('violacein_td.out');
plot(td.x_nm, td.eps_cont)      % UV-Vis spectrum
stem(td.nm, td.f)               % transition stem plot
td.trans{2}                     % transitions contributing to the 2nd
   state
```

> No G09 equivalent is provided: TD-DFT excited-state analysis and hyperpolarisability response properties in this toolbox are currently supported only for Gaussian 16 output files.

12.3 G16_td_plot

```
T = G16_td_plot(filename)
T = G16_td_plot(filename, 'Job', 2)                          % job of a
   --Link1-- chain
T = G16_td_plot(filename, 'XUnit', 'nm', 'FWHM', 0.4)
T = G16_td_plot(filename, 'States', [1 4 7], 'Overlay', true)
```

Reads the TD-DFT (or CIS) excited states of a Gaussian 16 log together with their orbital excitations, assigns a dominant excitation to every state and plots the spectrum. Compared with `G16_tddft`, it reads a chosen job of a `--Link1--` chain, keeps singlets and triplets apart, returns the weight $2c^2$ of every orbital excitation, labels orbitals relative to the frontier (`H-1->L+8`) and highlights a set of states with the same rule and colours used by `G16_td_levels`, so that the two figures of a calculation match.

The figure has two panels: the stick spectrum (f versus excitation energy; singlets, or all states of an unrestricted run, as stems; triplets, $f = 0$, as triangles on the axis), with the selected states in colour and labelled with state number, energy and dominant excitation; and, when `'FWHM'` > 0, the Gaussian-broadened absorption spectrum ε ($\mathrm{L\,mol^{-1}\,cm^{-1}}$) on the same axis. With `'Overlay', true` a single panel is drawn, with the broadened spectrum as a shaded envelope behind the sticks, scaled so that its maximum equals the largest f (shape only). A table of all states (number, multiplicity, eV, nm, f, main excitation) is printed in the Command Window.

Option	Default	Description
'Job'	last job with TD	Job of a --Link1-- chain to read (1-based)
'XUnit'	'eV'	'eV' | 'nm' | 'cm-1'
'FWHM'	0.3	Gaussian broadening FWHM in eV; 0 = sticks only
'States'	'auto'	States highlighted and labelled: 'auto' = first singlet + the 'NBright' most intense singlets up to 'EMax'; or a vector of state numbers
'NBright'	2	Number of intense states chosen by 'auto'
'EMax'	S_1 + 2.5 eV	Upper energy (eV) for 'auto'
'NLabel'	[]	Legacy option: label the NLabel most intense states instead
'Triplets'	true	Show the triplet states
'MinWeight'	0.05	Smallest weight $2c^2$ kept in .transitions
'Overlay'	false	Single panel with the scaled envelope behind the sticks
'Plot'	true	false = data only
'Title'	file, job	Figure title

Field	Description
.state, .mult, .sym	State number; multiplicity ('Singlet', 'Triplet', 'Doublet', ...); symmetry label
.E_eV, .nm, .cm1, .f, .S2	Excitation energy, wavelength, wavenumber, oscillator strength, $\langle S^2 \rangle$ (per state, in energy order)
.transitions	Cell array, one struct array per state: .from, .to, .coef, .weight (= $2c^2$ for closed shell), .deexc (true for <- lines), .label
.main	Per-state string with the dominant excitation, e.g. 'H->L+8 89%'
.HOMO	Index of the HOMO (number of α electrons)
.selected	State numbers highlighted (energy order)
.job, .filename	

```
T = G16_td_plot('NMA_wB97XD_augTZ_TD_trans.log', 'FWHM', 0.3);
tab = table(T.state', T.mult', T.E_eV', T.nm', T.f', T.main', ...
            'VariableNames', {'State','Mult','E_eV','nm','f','Main'})
T.transitions{5}                        % all excitations of S5 with
   weights
```

> The excited-state block read is the *last* one of the selected job: for TD optimisations or state-specific solvation (ExternalIteration) this is the final iteration. Orbital labels H/L refer to the α HOMO; in unrestricted runs the spin is appended. With diffuse basis sets the valence π^* orbital can lie well above the LUMO (e.g. LUMO+8 for N-methylacetamide in aug-cc-pVTZ), which the H/L labels make visible at once.

12.4 G16_td_levels

```
L = G16_td_levels(filename)
L = G16_td_levels(filename, 'Job', 1, 'States', [3 6 8], ...
```

```
                    'Labels', {19, '\pi_{nb}'; 20, 'n_O'; 29, '\pi^*'})
```

Draws the Kohn–Sham orbital-level diagram of one TD-DFT calculation around the frontier, with arrows for the dominant excitations of selected states. Occupied levels are dark grey and virtual levels light grey, thicker for HOMO, LUMO and the orbitals involved in the drawn excitations; energies are drawn at their true values and only the labels of near-degenerate levels are spread apart. For every selected state one arrow is drawn per excitation $i \to a$ with weight $2c^2 \geq$ `'MinWeight'`: one colour per state, line width growing with the weight, weight in % along the arrow and the state name (S1, S6, ...) at the tip. The legend gives energy, wavelength and oscillator strength of each state. The excited states are read with `G16_td_plot`, so the default selection and the colours are the same in the two figures.

Option	Default	Description
`'Job'`	last job with TD	Job of a `--Link1--` chain
`'States'`	`'auto'`	As in `G16_td_plot`
`'NBright'`, `'EMax'`	2, S_1 + 2.5 eV	As in `G16_td_plot`
`'MinWeight'`	`0.15`	Smallest weight drawn as an arrow
`'NOcc'`, `'NVirt'`	automatic	Occupied (HOMO included) and virtual (LUMO included) orbitals shown; default: enough for all drawn excitations, at least 3
`'Labels'`	{}	Chemical labels {`MO, 'label'; ...`}, TeX allowed
`'YLim'`	automatic	Energy range `[Emin Emax]` in eV
`'Title'`	file, job	Figure title
`'Plot'`	`true`	`false` = data only

Field	Description
`.HOMO`, `.occ_eV`, `.virt_eV`	HOMO index; all α occupied and virtual orbital energies (eV)
`.states`	State numbers drawn
`.arrows`	Struct array: `.state`, `.mult`, `.from`, `.to`, `.weight`, `.E_eV`, `.nm`, `.f`
`.file`, `.job`	

```
f = 'NMA_wB97XD_augTZ_TD_trans.log';
T = G16_td_plot(f);                       % spectrum: S1, S5, S10
    highlighted
L = G16_td_levels(f, 'Labels', {19,'\pi_{nb}'; 20,'n_O'; 29,'\pi^*'});
exportgraphics(gcf, 'trans_levels.pdf', 'ContentType', 'vector')
```

Requires `G16_td_plot.m` on the path. The orbital energies are the ground-state α eigenvalues printed after the last `Population analysis using the SCF density` of the selected job. Export the whole figure (`gcf`), not `gca`, to keep the legend.

13 Structural analysis utilities

13.1 G09_get_bond_length / G16_get_bond_length

```
bondTable = G09_get_bond_length(mol)
bondTable = G09_get_bond_length(mol, 'Name', Value, ...)
```

Builds the bond-length table of a molecule from a `mol` struct (fields `.symbols`, `.xyz`), detecting bonds via a covalent-radius criterion, and returns a MATLAB `table`.

Option	Default	Description
`'Tolerance'`	`1.15`	Multiplicative factor on the sum of covalent radii for the bonding criterion
`'IncludeH'`	`true`	Include bonds involving hydrogen atoms
`'SortBy'`	`'distance'`	Sort by `'distance'` (ascending) or `'atom'` (atom index)
`'SaveAs'`	`''`	Path (`.xlsx` or `.csv`) to save the table to; not saved if omitted

Output columns: `Atom1`, `Sym1`, `Atom2`, `Sym2`, `Distance_Ang`.

```
T = G09_get_bond_length(mol, 'SortBy', 'atom');
T = G16_get_bond_length(mol, 'IncludeH', false, 'SaveAs', 'bonds.xlsx
    ');
```

> The bonding criterion is purely geometric (covalent radii), not derived from Gaussian's own connectivity/topology matrix — suitable for quick structural analysis, not as the "official" G09/G16 connectivity.

13.2 G09_get_angle / G16_get_angle

```
[angTable, ang] = G09_get_angle(mol, atoms)
[angTable, ang] = G09_get_angle(mol, atoms, 'Name', Value, ...)
```

Bond angles *i*-*j*-*k* (degrees) from a molecular structure. `mol` is a struct with fields `.xyz` ($N\times3$, Å) and, optionally, `.symbols` (as returned by `G09_structure`/`G16_structure`, `G09_fchk_read`/`G16_fchk_read`, or `G_read_input`), or simply an $N \times 3$ coordinate matrix. `atoms` is an $M \times 3$ matrix of 1-based atom indices (Gaussian numbering); row m defines the angle `atoms(m,1)-atoms(m,2)-atoms(m,3)`, with the vertex in the middle. The angle is computed as atan2($|u \times v|$, $u \cdot v$) with $u = r_1 - r_2$, $v = r_3 - r_2$, which stays accurate near 0° and 180° (where `acos` loses precision).

Option	Default	Description
`'SaveAs'`	`''`	Path (`.xlsx` or `.csv`) to save the table to; not saved if omitted
`'Verbose'`	`false`	Print the table

Output columns: `Atom1`, `Sym1`, `Atom2`, `Sym2`, `Atom3`, `Sym3`, `Angle_deg` (symbols are `''` if `mol` has no `.symbols`).

```
mol = G16_structure('trans_optfreq.log');
G16_get_angle(mol, [4 7 8], 'Verbose', true)   % C4-N7=N8
T = G16_get_angle(mol, [4 7 8; 7 8 9]);        % both CNN angles
```

13.3 G09_get_dihedral / G16_get_dihedral

```
[dihTable, phi] = G09_get_dihedral(mol, atoms)
[dihTable, phi] = G09_get_dihedral(mol, atoms, 'Name', Value, ...)
```

Dihedral (torsion) angles *i*-*j*-*k*-*l* (degrees) from a molecular structure. `atoms` is an $M \times 4$ matrix of 1-based atom indices; row m defines the dihedral `atoms(m,1)-(m,2)-(m,3)-(m,4)`, i.e. the angle between the planes $(1,2,3)$ and $(2,3,4)$ seen along the central bond 2–3. With $b_1 = r_2 - r_1$, $b_2 = r_3 - r_2$, $b_3 = r_4 - r_3$, and plane normals $n_1 = b_1 \times b_2$, $n_2 = b_2 \times b_3$:

$$\phi = \operatorname{atan2}\Big((n_1 \times n_2) \cdot \tfrac{b_2}{|b_2|},\ \ n_1 \cdot n_2 \Big).$$

The dihedral is undefined (`NaN`, with a warning) if three consecutive atoms are collinear (e.g. at a linear CNN angle).

Option	Default	Description
`'Signed'`	`true`	`true`: signed dihedral in $(-180, 180]$, IUPAC convention (positive = clockwise rotation of the front bond 1-2 onto the back bond 3-4 seen along 2 → 3); `false`: absolute value in $[0, 180]$
`'SaveAs'`	`''`	Path (`.xlsx` or `.csv`) to save the table to; not saved if omitted
`'Verbose'`	`false`	Print the table

Output columns: `Atom1`, `Sym1`, ..., `Atom4`, `Sym4`, `Dihedral_deg`.

```
mol = G16_structure('cis_optfreq.log');
G16_get_dihedral(mol, [4 7 8 9], 'Verbose', true)       % CNNC
[~, phi] = G16_get_dihedral(mol, [4 7 8 9], 'Signed', false);
```

> Both functions are version-agnostic like `G_read_input`/`G_spectra_nm` (identical G09/G16 copies), but keep the `G09_`/`G16_` naming convention of the rest of the toolbox rather than a shared `G_` helper, matching `get_bond_length`/`draw_molecule` just above.

13.4 G09_nbo_bonds / G16_nbo_bonds

```
bt = G09_nbo_bonds(filename)
bt = G09_nbo_bonds(filename, 'Name', Value, ...)
```

Determines bond order (single/double/triple) from an actual Gaussian NBO (Natural Bond Orbital) analysis, as opposed to `get_bond_length`'s geometric covalent-radius criterion or `draw_molecule`'s bond-length heuristic. Reads the "Natural Bond Orbitals (Summary)" table and counts, for every atom pair, how many bonding (`BD`) natural bond orbitals NBO assigned to it: one `BD` orbital means a single bond (σ only), two mean a double bond ($\sigma+\pi$), three mean a triple bond ($\sigma+2\pi$). Antibonding (`BD*`) orbitals are excluded.

Option	Default	Description
'section'	'last'	Which "Natural Bond Orbitals (Summary)" block to read, for files with more than one (e.g. multi-step opt+freq+polar jobs that repeat the NBO analysis at each step); also accepts 'first'
'Lines'	{}	Pre-read cell array of file lines, to skip re-reading the file

Output columns: Atom1, Sym1, Atom2, Sym2, BondOrder, Occupancy (summed NBO occupancy, ≈ 2 per BD orbital).

```
bt = G09_nbo_bonds('CH4_NBO.LOG');
```

> Requires the source file to have been computed with the pop=nbo Gaussian keyword (or similar, e.g. pop=(nbo,savenbo)); without it, the output file simply does not contain NBO data and the function raises an error — bond order cannot be recovered after the fact from geometry/energy alone. This reads the "Natural Bond Orbitals (Summary)" table that pop=nbo always prints, not the separate "Wiberg bond index matrix" (which additionally requires pop=(nbo,bndidx) and is not parsed by this function).

13.5 G09_gaussian_version / G16_gaussian_version

```
gv = G09_gaussian_version(filename)
gv = G16_gaussian_version(filename)
```

Detects which Gaussian version/revision produced a .out/.log/.fchk file.

> For .out/.log files, reads the "Gaussian NN, Revision X.YY," citation line printed near the top of the file (works for any NN: 09, 16, ...). .fchk files do not store this information at all; in that case the function looks for a sibling .log/.out file with the same base name in the same folder and reads the version from there. If no sibling file is found, .major/.revision/.full are returned empty and a warning is issued.

Field	Description
.major	Gaussian major version, e.g. 9, 16 ([] if unknown)
.revision	Revision string, e.g. 'A.02', 'C.01' ('' if unknown)
.full	Citation line as printed, e.g. 'Gaussian 09, Revision A.02'
.source	'out/log' | 'fchk-sibling:<path>' | 'unknown'
.filename	

```
gv = G09_gaussian_version('a1.out');         % gv.major = 16, gv.
    revision = 'C.01'
gv = G16_gaussian_version('molecule.fchk'); % falls back to molecule.
    out/.log if present
```

13.6 G09_restart / G16_restart

```
gjf_file = G09_restart(filename)
gjf_file = G09_restart(filename, 'Name', Value, ...)
```

```
gjf_file = G16_restart(filename)
gjf_file = G16_restart(filename, 'Name', Value, ...)
```

Generates a Gaussian 09/16 input file (`.gjf`) to restart a calculation from a geometry found in an existing `.log`/`.out` file. `G16_restart` reads geometry via `G16_structure` (so it follows that function's Standard/Input orientation handling); otherwise the two functions are identical.

Option	Default	Description
`'step'`	`'last'`	`'last'` | `'first'` | integer N — which geometry step to restart from
`'output'`	`<base>_restarted.gjf`	Output `.gjf` path
`'extra_route'`	`''`	Additional keywords appended to the route section
`'nproc'`	`0`	`%nprocshared` override (`0` = keep/omit)
`'mem'`	`''`	`%mem` override (`''` = keep/omit)

> If the source output file has no `%chk` line (common in Gaussian 09 Windows output), the checkpoint name is derived from the source file name instead. The generated `.gjf` is valid for restarting under either Gaussian 09 or Gaussian 16, and can be read back with `G_read_input`.

Returns the path of the generated `.gjf` file.

13.7 G09_available_data / G16_available_data

```
T = G09_available_data(filename)
T = G16_available_data(filename)
```

Reads the route section (see `G09_route`/`G16_route`) and checks it against the keyword requirements documented in Section 2 ("Which `.in` keywords generate which `.out` section"), so you can tell up front which data will actually be present in a file — e.g. a TD-DFT-only single point job (`td=(singlets,nstates=100)`, no `freq`) has no vibrational normal modes, no IR/Raman spectra, and no thermochemistry (ZPE/H/G/S), so calling `nmodes`/`spectra` on it raises an error and `energy`'s `*_corr`/`E0`/`U`/`H`/ `G`/`T`/`P`/`S` fields come back `NaN` — none of that is a bug, it is simply not in the file. This function lets you check that in one call, rather than discovering it one error/NaN at a time.

Field	Description
`.Function`	Toolbox function/quantity name
`.Available`	`logical` — true if the route contains the keyword(s) needed
`.Requires`	Human-readable keyword requirement (e.g. `'freq'`, `'polar + cphf=rdfreq'`)
`.Notes`	Short clarifying note

```
G16_available_data('V_E00_R_24_TD.out');
T = G16_available_data('molecule.out');
T(~T.Available, :)   % see what's NOT available in this file
```

This predicts availability from the route keywords alone; it does not itself read the data sections, so an incomplete/crashed job can still show a keyword as present without the corresponding output actually having been printed (the extraction function will still raise its own clear error in that case). `G09_available_data` omits the `hyperpolar/tddft` rows entirely, since those functions do not exist in the G09 toolbox.

13.8 G_read_input (shared, no G09_/G16_ prefix)

```
ginp = G_read_input(filename)
```

Reads a Gaussian *input* file (`.gjf`, `.com`, or `.in`) and extracts the link0 commands, route section, title, charge/multiplicity, and starting Cartesian geometry.

The Gaussian input file format does not differ between Gaussian 09 and Gaussian 16, so this is the toolbox's only function without a `G09_/G16_` prefix pair: an identical copy is kept in both the `G09/` and `G16/` folders, so it is available regardless of which toolbox you have on the MATLAB path. Only Cartesian-coordinate geometry blocks are supported (not Z-matrix input).

Field	Type	Description
`.symbols`	cell	Atomic symbols
`.xyz`	double	Starting coordinates in Å (X Y Z)
`.Natoms`	int	Number of atoms
`.charge`	int	Total molecular charge
`.mult`	int	Spin multiplicity
`.title`	char	Title/comment line(s), joined with a space
`.route`	char	Full route section, single line
`.method`	char	Method parsed from the route (e.g. `'b3lyp'`), `''` if not found
`.basis`	char	Basis set parsed from the route (e.g. `'6-311+g(d,p)'`), `''` if not found
`.chk`	char	`%chk` link0 value, `''` if absent
`.mem`	char	`%mem` link0 value, `''` if absent
`.nproc`	char	`%nprocshared`/`%nproc` link0 value, `''` if absent
`.filename`	char	Source file path

`ginp` has the same `.symbols`/`.xyz`/`.Natoms`/ `.filename` fields as the struct returned by `G09_structure`/`G16_structure`, so it can be passed directly to `G09_draw_molecule`/`G16_draw_molecule` or `G09_get_bond_length`/`G16_get_bond_length` without any conversion.

```
ginp = G_read_input('molecule.gjf');
G16_draw_molecule(ginp, 'ShowAxes', true);
fprintf('%s/%s, charge %d, mult %d\n', ginp.method, ginp.basis, ...
    ginp.charge, ginp.mult);
```

13.9 G09_list / G16_list

```
G09_list()
T = G09_list()

G16_list()
T = G16_list()
```

Lists every `G09_*.m`/`G16_*.m` function found in the toolbox folder, printing the function name and its H1 description line (sorted alphabetically). Also returns the same information as a `table` with columns `.Name`, `.Description`, `.File`.

```
G09_list();
T = G16_list();
T(contains(T.Description, 'dipole', 'IgnoreCase', true), :)
```

14 One-call data extraction

14.1 G09_read_all / G16_read_all

```
T = G09_read_all(filename)
T = G16_read_all(filename)
```

Runs the full set of `G09_XXX`/`G16_XXX` parsers on a single output file and collects the results into one struct `T`. The file is read from disk once and the resulting lines are passed to every sub-parser via their `'Lines'` option, instead of each sub-parser independently re-reading and re-splitting the file.

Field	Source function	Description
`.charge`	`G09/G16_charges`	Mulliken and APT atomic charges (plotting disabled)
`.energy`	`G09/G16_energy`	SCF energy and thermochemistry
`.structure`	`G09/G16_structure`	Optimized molecular structure
`.dipolar`	`G09/G16_dipole_polar`	Dipole moment and polarisability
`.nmodes`	`G09/G16_nmodes`	IR/Raman vibrational normal modes; omitted (with a non-blocking warning) if the file has no frequency calculation — check with `isfield(T, 'nmodes')`
`.spectra`	`G09/G16_spectra`	Frequencies, IR/Raman intensities, simulated spectra; omitted under the same condition as `.nmodes`
`.route`	`G16_route` (G16 only)	Gaussian route section details
`.chargemol.charge`, `.chargemol.mol`	`G16_charge_mult` (G16 only)	Total molecular charge, spin multiplicity

```
T = G16_read_all('zeatin.out');
disp(T.energy)
G16_draw_molecule(T.structure);
```

Known discrepancy: the current `G09_read_all` does *not* populate `.route` or `.chargemol`, unlike `G16_read_all`, which retains both. If parity with `G16_read_all` is required, add

```
T.route = G09_route(filename, 'Lines', lines);
[c, m]  = G09_charge_mult(filename, 'Lines', lines);
T.chargemol.charge = c;
T.chargemol.mol    = m;
```

to `G09_read_all.m`.

Bug fixed (2026-08-05): `G09_read_all`/`G16_read_all` (and the Python `g16_read_all`) used to call `nmodes`/`spectra` unconditionally, so a job with no frequency calculation at all (e.g. a TD-DFT-only single point job, `td=(singlets,nstates=100)` with no `freq` keyword) crashed the *entire* call, even though every other field (charges, energy, structure, dipole, route, charge/multiplicity) would have extracted perfectly fine. This directly contradicted `G09_write_report.m`/`G16_write_report.m` (and their Python counterpart), which already used `isfield`/`getattr` checks expecting `.nmodes`/`.spectra` to be *option-*

ally absent. Fixed by wrapping just those two calls in `try/catch`: on failure the field is omitted entirely (not left as an empty placeholder) and a single non-blocking `warning(...)` explains why, restoring the graceful-degradation behaviour the rest of the toolbox already assumed.

14.2 G09_batch_read_all / G16_batch_read_all

```
T = G09_batch_read_all(folder)
T = G09_batch_read_all(folder, 'Recursive', true)
T = G09_batch_read_all(folder, 'WriteReports', true)
T = G09_batch_read_all(folder, 'SaveAs', 'summary.xlsx')

T = G16_batch_read_all(folder)
```

Scans `folder` for `.log`/`.out` files (case-insensitive), runs `G09_read_all`/`G16_read_all` (plus `G09_orbital_energies`/`G16_orbital_energies`, for the HOMO-LUMO gap) on each, and returns one row per file — useful for screening many calculations at once (conformers, scans, batches of jobs) without calling `read_all` file-by-file.

A file that fails to parse (e.g. an incomplete/crashed job, or a file from the other Gaussian version run through the wrong toolbox) does *not* stop the batch: its row is filled with `NaN` and the caught error message is recorded in the `.Status` column instead.

Option	Default	Description
`'Recursive'`	`false`	Also scan subfolders
`'WriteReports'`	`false`	Write a `G09_write_report`/`G16_write_report` `.txt` next to each source file
`'SaveAs'`	`''`	Path to save the summary table (`.csv` or `.xlsx`, inferred from the extension)

Field	Description
`.File`	Source file path
`.Natoms`	Atom count (from `.structure.Natoms`)
`.SCF_Hartree`	SCF energy
`.E0_Hartree`	SCF + ZPE
`.G_kJmol`	Gibbs free energy, kJ/mol
`.mu_tot`, `.mu_units`	Dipole magnitude and its units
`.HOMO_eV`, `.LUMO_eV`, `.Gap_eV`	Frontier orbital energies and gap, eV
`.Status`	`'ok'`, or the caught error message if parsing failed

```
T = G16_batch_read_all('results/', 'WriteReports', true);
T(T.Gap_eV == min(T.Gap_eV), :)   % smallest HOMO-LUMO gap
```

15 Report generation

15.1 G09_write_report / G16_write_report

```
outfile = G09_write_report(T)
outfile = G09_write_report(T, outfile)

outfile = G16_write_report(T)
outfile = G16_write_report(T, outfile)
```

Writes a human-readable text report from a `G09_read_all`/ `G16_read_all` struct `T`, formatting a summary of every field present in `T` into a plain `.txt` file. If `outfile` is omitted, the file is named after the source Gaussian file (e.g. `'zeatin.out'` → `'zeatin_report.txt'`) in the current folder.

Argument	Default	Description
`T`	(required)	Struct returned by `G09_read_all`/`G16_read_all`
`outfile`	`<source>_report.txt`	Output `.txt` path

Returns the path written to. Sections are included only if the corresponding field is present in `T`, in this order: route, charge/multiplicity, molecular structure (full Cartesian coordinate table), energetics (with thermochemistry if available), dipole moment and polarisability (including the dynamic polarisability table, if present), atomic charges (full per-atom table, sum of charges, dipole-from-charges overlay if computed), vibrational normal modes (frequency/IR/Raman/reduced mass/symmetry table), and a summary of the simulated IR/Raman spectra.

> The full broadened-spectrum continuum arrays (`T.spectra.x`, `.IR_cont`, `.Raman_cont`) are deliberately *not* dumped into the report — only the grid range, FWHM, and whether Raman data is present are summarised. Access those arrays directly from `T.spectra` for plotting or export.

```
T = G16_read_all('zeatin.out');
G16_write_report(T);                          % -> zeatin_report.txt
G16_write_report(T, 'zeatin_summary.txt');    % explicit output path
```

No differences between the G09 and G16 versions beyond the input struct's origin and the report header ("Gaussian 09"/"Gaussian 16 Calculation Report").

See also `G09_read_all`/`G16_read_all`.

`G16parser` is a Python 3 port of the G16 MATLAB toolbox, covering the same parsing, checkpoint-analysis, and static-visualisation functionality described in the previous sections. It is installable as a standard editable package and exposes one `g16_xxx` function per MATLAB `G16_xxx` counterpart, each returning a `Struct` object (attribute access, e.g. `mol.xyz`, `ch.dipole_Debye`) instead of a MATLAB struct.

```
cd G16parser
pip install -e .
```

```
import G16parser as g16

mol = g16.g16_structure('molecule.out')
g16.g16_draw_molecule(mol, show_axes=True)

ch = g16.g16_charges('molecule.out', show_dipole=True)
```

```
oe = g16.g16_orbital_energies('molecule.out')
g16.g16_draw_orbital(oe)

T = g16.g16_read_all('molecule.out')   # everything in one call,
    single file read
```

> **Requirements:** Python ≥3.9, `numpy`, `pandas`, `matplotlib` (installed automatically via `pip install -e .`). No Gaussian installation or compiled dependency is needed.

15.2 Function reference

Every function mirrors its MATLAB `G16_xxx` counterpart in name, parameters (in `snake_case`), defaults, and returned fields — see the corresponding MATLAB subsection earlier in this manual for full parameter/field documentation. All extraction functions accept an optional `lines=` parameter (pre-read file lines) mirroring the MATLAB `'Lines'` option; `g16_read_all` uses this internally to read the file exactly once.

Function	Description
`g16_au_convert`	Converts a physical quantity between atomic units and SI/another common unit (§8.6)
`g16_au_table`	Prints a formatted reference table of atomic-unit conversion factors (CODATA 2022)
`g16_gaussian_field_convert`	Converts between Gaussian's `Field=X+N` route keyword and a physical field strength
`g16_read_input`	Reads a Gaussian *input* file (`.gjf`/`.com`/`.in`): link0 commands, route, title, charge/multiplicity, and starting geometry — output `Struct` is compatible with `g16_draw_molecule`/`g16_get_bond_length`
`g16_structure`	Molecular geometry (`symbols`, `xyz` as `np.ndarray`, atom count)
`g16_energy`	SCF energy and thermochemistry
`g16_convergence`	Geometry-optimisation convergence criteria per step (`plot=True` for a matplotlib figure)
`g16_charges`	Mulliken/APT atomic charges, optional dipole overlay and 3D plot
`g16_dipole_polar`	Dipole moment and (static/dynamic) polarisability
`g16_nmodes`	Vibrational normal-mode displacement vectors
`g16_spectra`	IR/Raman spectra (stick + Lorentzian-broadened continuum)
`g16_spectra_nm`	Same Lorentzian-broadened IR/Raman spectra as `g16_spectra`, built directly from an already-parsed `nm` struct (from `g16_nmodes`, or the `.nm` field of `g16_fchk_read`) — no `.log`/`.out` file needed
`g16_orbital_energies`	HOMO/LUMO and the full occupied/virtual MO spectrum
`g16_charge_mult`	Molecular charge and spin multiplicity
`g16_route`	Route section string
`g16_available_data`	Lists which toolbox quantities are expected to be extractable from a file, by checking its route keywords (e.g. "no `freq` keyword → no nmodes/spectra/thermochemistry") — catches this before an error or a NaN
`g16_get_bond_length`	Bond-length table (pandas DataFrame) from covalent radii
`g16_get_angle`	Bond angle *i*-*j*-*k* (degrees) table from a molecular structure
`g16_get_dihedral`	Dihedral (torsion) angle *i*-*j*-*k*-*l* (degrees) table from a molecular structure
`g16_nbo_bonds`	Bond order (single/double/triple) from an actual Gaussian NBO analysis (`pop=nbo`), by counting bonding (BD) natural bond orbitals per atom pair
`g16_fchk_read`	Reads a Gaussian formatted checkpoint (`.fchk`) file — works on both G09 and G16 `.fchk` files, since the format does not depend on the Gaussian version
`g16_charges_fchk`	Visualises charges from a `g16_fchk_read` struct, without needing the corresponding `.log`/`.out` file
`g16_gaussian_version`	Detects the Gaussian version/revision (works on `.fchk` via a sibling `.log`/`.out`)
`g16_hyperpolar`	Dipole hyperpolarisability (β)
`g16_tddft`	TD-DFT excited states
`g16_split_link1_log`	Splits a `--Link1--` chained log into one log per job; names from `%chk` or from a field value in the title (no G09/G16 distinction: the `--Link1--` chaining format is version-independent)

15.3 Selected signatures

```
g16_au_convert(value, quantity, direction, target="", verbose=None)
g16_au_table(section="")
g16_gaussian_field_convert(value, direction, unit="au", verbose=True)
g16_read_input(filename)
g16_available_data(filename)
g16_structure(filename, orientation="auto", step="last", lines=None)
g16_energy(filename, step="last", lines=None)
g16_convergence(filename, plot=False, lines=None)
g16_nmodes(filename, section="last", modes=None, lines=None)
g16_spectra(filename, FWHM=10, xmin=0, xmax=4000, dx=1, normalize=
    False,
            plot=False, section="last", lines=None)
g16_spectra_nm(nm, FWHM=10, xmin=0, xmax=4000, dx=1, normalize=False,
    plot=False)
g16_charges(filename, type="Mulliken", mode="atom", plot=True,
            atom_scale=0.35, bond_tol=1.30, bond_list=None, font_size
                =8,
            color_scale="RdBu",
            threshold=0.0, show_dipole=False, dipole_origin="negcharge
                ",
            dipole_scale=1.0, dipole_color=(0, 0.6, 0),
                dipole_line_width=2.5,
            show_dipole_label=True, dipole_font_size=11, dipole_units
                ="Debye",
            lines=None)
g16_draw_molecule(mol, atom_scale=0.35, bond_tol=1.30, show_labels=
    True,
                  show_legend=True, title="", bg_color=(0.95, 0.95,
                      0.95),
                  ax=None, show_axes=False, axes_length=None,
                      bond_list=None)
g16_draw_deformation(mol1, mol2, overlay=False, scale=1, arrow_color
    =(1.0, 0.4, 0.1),
                     overlay2_color=(0.75, 0.1, 0.1), atom_scale=0.35,
                         bond_tol=1.30,
                     show_labels=False, arrow_threshold=0.05)
g16_animate_mode(mol, nm, mode_idx, filename=None, scale=1.5,
    flip_sign=False,
                 atom_scale=0.35, bond_tol=1.30, show_labels=False,
                 frames_per_cycle=30, n_cycles=2, fps=20, view=None,
                 dpi=150, progress_callback=None,
                 crop=True, crop_margin=12, show_title=False)
g16_mode_viewer(filename, scale=1.5, atom_scale=0.35, bond_tol=1.30,
                show_labels=False, flip_sign=False, arrow_color=None,
                font_size=10, **extra_opts)
g16_get_bond_length(mol, tolerance=1.15, include_h=True,
                    sort_by="distance", save_as="")
g16_get_angle(mol, atoms, save_as="", verbose=False)
g16_get_dihedral(mol, atoms, signed=True, save_as="", verbose=False)
g16_nbo_bonds(filename, section="last", lines=None)
g16_fchk_read(filename, verbose=True)
g16_charges_fchk(mol, ch, mode="atom", plot=True, atom_scale=0.35,
                 bond_tol=1.30, font_size=8, color_scale="RdBu",
                    threshold=0.0)
g16_hyperpolar(filename, units="au", lines=None)
g16_tddft(filename, nstates=math.inf, plot=False, fwhm_ev=0.30, lines=
    None)
```

```
g16_split_link1_log(logfile, naming="auto", pattern=r"scan along X: N
    = (-?\d+)",
                        prefix="", outdir="")
g16_restart(filename, step="last", output="", extra_route="", nproc=0,
     mem="")
g16_batch_read_all(folder, recursive=False, write_reports=False,
    save_as="")
g16_write_report(T, outfile=None)
```

15.4 Notes on the port

Test suite. A `pytest` suite lives in `G16parser/tests/` (`pip install -e ".[test]"` then `pytest` from the `G16parser/` folder). Most tests need one real Gaussian 16 `.out`/`.log` file dropped into `tests/fixtures/` (any filename); without one, those tests are skipped rather than failed, so the suite stays runnable out of the box. `tests/fixtures/water.gjf` is a small hand-written synthetic input file (not from a real calculation) used for the `g16_read_input` tests, which do not depend on the optional real-output fixture.

Continuous integration. The test suite runs automatically on every push/pull request to `main` via GitHub Actions (`.github/workflows/python-tests.yml`), against Python 3.9 and 3.12 on Ubuntu. Coverage is Python-only: running the MATLAB `G09_*.m`/`G16_*.m` functions in CI would additionally require a valid MATLAB licence (MathWorks' `matlab-actions/setup-matlab` GitHub Action needs one even for public repositories), so the MATLAB side of the toolbox continues to be verified manually.

3D interactivity. matplotlib's `mplot3d` has no MATLAB-style `rotate3d` widget; use the interactive backend's normal mouse/toolbar controls instead.

Naming exception: `g16_read_input`. On the MATLAB side this function has no version prefix at all (`G_read_input`, identical copies in both the `G09/` and `G16/` folders) because the Gaussian input file format does not differ between versions. Since only a `G16parser` Python package exists (there is no separate `G09parser`), the port simply follows this package's `g16_` naming convention like every other function here.

g16_restart and **g16_batch_read_all** port `G16_restart.m` and `G16_batch_read_all.m` only — there is no Python equivalent of `G09_restart.m`/`G09_batch_read_all.m`, consistent with every other G09-only MATLAB function (e.g. `G09_read_lines`, an internal helper with no G16 counterpart since `G16_*.m` functions inline the equivalent file-reading logic instead), since this package only targets Gaussian 16 output. Note that `G09_fchk_read`/`G09_charges_fchk` are *not* G09-only despite the prefix (identical `G16_` counterparts exist, since the `.fchk` format itself does not depend on the Gaussian version) — both were ported to Python (`g16_fchk_read`/`g16_charges_fchk`) alongside their `G16_` counterparts.

`g16_animate_mode`'s frame cropping is implemented differently from `G09_animate_mode.m`/`G16_animate_mode.m`. MATLAB captures each rendered frame's pixels directly (`getframe`) and crops that array in memory before writing it to

`VideoWriter`. matplotlib's `FFMpegWriter` offers no equivalent per-frame pixel hook, so the Python port instead (1) renders the two oscillation-extreme probe frames to compute the same tight content bounding box (padded by `crop_margin`, evened for `yuv420p`), (2) writes the full, uncropped video to a temporary file exactly as without `crop`, then (3) runs a single `ffmpeg -vf crop=W:H:X:Y` pass over it to produce the final, cropped file. This relies on nothing beyond `ffmpeg` itself (already a hard requirement for MP4 output at all), avoiding both a new dependency and any private/internal matplotlib API. Verified directly: cropped output is consistently smaller (in both dimensions, always even) than the uncropped video of the same mode, and enabling `show_title` measurably enlarges the crop's height, matching the MATLAB behaviour.

Backend selection. The package forces the `TkAgg` matplotlib backend (if Tk is available and `pyplot` has not been imported yet) instead of the native macOS backend: on some older matplotlib releases, interactively rotating a 3D plot with the native Cocoa backend can segfault the whole Python process. Call `matplotlib.use(...)` yourself *before* importing `G16parser` if a different backend is needed.

Bug found during porting, MATLAB-side only (fixed 2026-07-13): in an early version of `G16_tddft.m`, MATLAB's `regexp` silently dropped the optional trailing `<S**2>=` capture group even when it matched (a MATLAB-specific regex-engine quirk), so `td.S2` was always `NaN` in MATLAB even when the tag was present in the file. This Python port's `re`-based parser never shared that bug. Rather than leaving this as a permanent discrepancy between the two implementations, the MATLAB-side parser was fixed to extract `<S**2>=` with its own independent `regexp` call, instead of a trailing optional group; both implementations now agree.

Bug found during porting, MATLAB-side only (fixed 2026-08-03): the IR/Raman intensity calculation in `G09_fchk_read.m`/ `G16_fchk_read.m` re-reshaped the already-correctly-shaped dipole/ polarisability derivative matrices a second time (`reshape(dip_deriv, N3, 3)'`) before use. Since MATLAB's `reshape` reinterprets the same column-major buffer under the new shape rather than transposing back to the original layout, this silently scrambled which derivative value belonged to which atom/displacement, producing plausible-looking but wrong `nm.IR`/`nm.Raman` values — caught by comparing a `.fchk`-derived spectrum against the same molecule's IR/Raman as printed directly by Gaussian in the corresponding `.out` file. Fixed in both MATLAB files by using the matrices directly (no second reshape needed). This Python port's `g16_fchk_read` never had the bug (written directly with the correct indexing) — verified to match the `.out`-printed values, and a regression test (`test_fchk_ir_raman_matches_out_ground_truth`) now guards against it ever being reintroduced.

`g16_spectra_nm` ports `G_spectra_nm.m` (Section 4.9), a genuinely version-agnostic MATLAB function (no `G09_`/`G16_` prefix, like `G_read_input`): it builds the same Lorentzian-broadened IR/Raman continuum as `g16_spectra`, but directly from an already-parsed `nm` Struct (from `g16_nmodes`, or the `.nm` field of `g16_fchk_read`) instead of reading a `.log`/`.out` file — useful when only a `.fchk` file is available. Verified to match `g16_spectra` called directly on the corresponding `.out` file for the same molecule.

g16_available_data ports `G09_available_data.m`/ `G16_available_data.m` (Section 13.7). Unlike the MATLAB `G09_` version, it never omits the `hyperpolar`/`tddft` rows, since this package only targets Gaussian 16 output and both functions always exist here.

Bug fixed (2026-08-05), both MATLAB and Python: `g16_read_all` (and `G09_read_all.m`/`G16_read_all.m`) used to call `g16_nmodes`/`g16_spectra` unconditionally, so a file with no frequency calculation at all (e.g. a TD-DFT-only single point job) raised `ValueError` and crashed the entire call, even though every other field would have extracted fine. Fixed by wrapping just those two calls in `try`/`except`: on failure, `nmodes`/`spectra` are set to `None` (checked via `getattr(T, "nmodes", None)`, matching how `g16_write_report` already consumed them) and a single `warnings.warn(...)` explains why.

g16_mode_viewer is a Tkinter rewrite of `G16_modeViewer.m`'s `uifigure` GUI, with the same controls (mode selector, order-by, per-option redraw, title, save-as PDF/EPS/JPEG, and an "Animate mode (MP4)..." button wired to `g16_animate_mode`, using the current option values and the displayed mode figure's camera orientation). One simplification: "target figure" always means the most recently drawn mode window, not whichever window the user last clicked on (MATLAB's `groot().CurrentFigure` has no simple Tkinter equivalent). In an IPython/Spyder console with the Tk graphics backend active, the viewer detects the running event loop and returns to the prompt immediately instead of blocking; run as a plain script, it blocks until the viewer window is closed.
Known issue: `g16_mode_viewer` can segfault when run from *inside* Spyder's IPython console (a crash inside ipykernel's own periodic Tk event-loop pump, not in this toolbox's code) — confirmed working correctly when run from a plain terminal/script instead:

```
python3 -c "import G16parser as g16; g16.g16_mode_viewer('file.out
    ')"
```

All other functions, including static plots, work fine inside Spyder.

g16_animate_mode mirrors `G09_animate_mode.m`/ `G16_animate_mode.m`: it oscillates the molecule along the mode's displacement vector and saves an MP4 via `matplotlib.animation.FuncAnimation` with `FFMpegWriter`. **Requires `ffmpeg` installed and on `PATH`** (`brew install ffmpeg` on macOS, `sudo apt install ffmpeg` on Ubuntu/Debian) — matplotlib does not bundle a video encoder itself; without it the call fails with `FileNotFoundError: ... 'ffmpeg'` at the final encoding step (frame generation itself does not need ffmpeg). Bonds are computed once from the equilibrium geometry via `g16_get_bond_length` and passed to `g16_draw_molecule` through its `bond_list` parameter, so they stay fixed across all frames instead of flickering as instantaneous distances cross `bond_tol`. By default the animation uses matplotlib's default 3D view; pass `view=(ax.azim, ax.elev)` to start from an already-rotated figure's orientation instead.

g16_write_report takes the `Struct` returned by `g16_read_all` and writes the same section-by-section plain-text report as `G09_write_report.m`/`G16_write_report.m` (route, charge/multiplicity, structure, energetics, dipole/polarisability, atomic charges, normal modes, spectra summary), using Python `getattr`-based field checks so that sections are included only when present, exactly mirroring the MATLAB `isfield` logic.

Bond order (single/double/triple). `g16_draw_molecule` mirrors `G09_draw_molecule.m/G16_draw_molecule.m`: for C-C and C-O pairs, bond order is estimated purely from bond length (any other element pair, including C-N, is always drawn as a single bond) and rendered as 1/2/3 parallel lines. This is a geometric estimate, not Gaussian's own bond-order analysis (e.g. Wiberg/NBO indices). For an actual bond order derived from Gaussian's own NBO analysis, use `g16_nbo_bonds` (Section 13.4). The `bond_list` parameter accepts an optional 3rd column with a pre-computed order, used by `g16_animate_mode` (via `g16_get_bond_length`) to keep both bond topology and bond order fixed across all animation frames:

```
mol = g16.g16_structure('molecule_nbo.out')
bt  = g16.g16_nbo_bonds('molecule_nbo.out')   # requires 'pop=nbo'
   in the source job

bond_list = bt[['Atom1', 'Atom2', 'BondOrder']].to_numpy()
bond_list[:, :2] -= 1   # g16_nbo_bonds uses 1-based Gaussian atom
   numbers;
                        # g16_draw_molecule's bond_list is 0-based
g16.g16_draw_molecule(mol, bond_list=bond_list)
```

`g16_charges` accepts the same `bond_list=` parameter, passed straight through to its internal `g16_draw_molecule` call:

```
g16.g16_charges('molecule_nbo.out', bond_list=bond_list,
   show_dipole=True)
```

The C-C double/single threshold is set to 1.36 Å rather than the generic reference-length midpoint, so symmetric aromatic rings (C-C ≈1.39–1.40 Å, no real length alternation) render as all-single instead of all-double. C-N is deliberately excluded from length-based classification: on asymmetric pyridine-like rings a single threshold produced one ring C-N bond rendered double and its chemically-equivalent neighbour rendered single — see the MATLAB `G09_draw_molecule/G16_draw_molecule` subsection earlier in this manual for the full rationale.

16 Worked examples

Four self-contained scripts in `G16/` (`example_1.m`, `example_2.m`, `example_3.m`, `example_4.m`) demonstrate the toolbox end to end on a single real DFT calculation: `test_2.out`, a geometry optimisation of Violacein (G. Cassone et al., *Phys. Chem. Chem. Phys.*, doi: 10.1039/d6cp01164k). Each script only needs `G16/` on the MATLAB path and `test_2.out` in the current folder.

16.1 example_1.m — Structure, vibrational mode, and charges

Covers `G16_structure/G16_draw_molecule` (3D structure rendering), `G16_nmodes/G16_draw_mode` (a single vibrational mode, #91, the main C=C stretch), and `G16_charges` (Mulliken charges with a dipole moment overlay), all rendered from the same camera angle and each exported to its own PDF.

```
%% G16 Toolbox -- Example #1: structure, vibrational mode, and charges
%
%   Demonstrates three core G16_*.m functions on a real DFT
   calculation:
%     1) G16_structure  + G16_draw_molecule  -- 3D structure rendering
%     2) G16_nmodes     + G16_draw_mode      -- a single vibrational
   mode
%     3) G16_charges                         -- Mulliken charges +
   dipole
%
%   Data file: test_2.out -- DFT geometry optimisation of Violacein.
%   Reference: G. Cassone et al., Phys. Chem. Chem. Phys.,
%              doi: 10.1039/d6cp01164k
%
%   Requirements: G16/ on the MATLAB path, test_2.out in the current
%   folder. Running this script creates three PDFs in the current
%   folder: test_2_mol.pdf, test_2_mode_91.pdf, test_2_charges.pdf.

clear
close all
clc

filename = 'test_2.out';

% Camera orientation shared by all three figures below, so the
   molecule
% is viewed from the same angle in every plot.
view_az = -16.4417;
view_el =  44.0197;

%% 1) Structure -- load the optimised geometry and render it in 3D
mol = G16_structure(filename);

% Create the axes up front so it can be tweaked (view angle, export)
% after G16_draw_molecule has finished drawing into it.
ax1 = axes;
set(ax1, 'Visible', 'off');
G16_draw_molecule(mol, 'Title', 'Violacein', 'Ax', ax1);
set(ax1, 'View', [view_az, view_el]);

% ContentType 'vector' gives a crisp publication figure but is slow
   for
% a molecule this size; use 'image' instead for a quick low-effort
   preview.
```

```
exportgraphics(ax1, 'test_2_mol.pdf', 'ContentType', 'vector');

%% 2) Vibrational mode -- mode #91 is the main C=C stretch
nm = G16_nmodes(filename);

ax2 = axes;
set(ax2, 'Visible', 'off');
G16_draw_mode(mol, nm, 91);
set(ax2, 'View', [view_az, view_el]);

% Replace only the molecule-name line of the auto-generated two-line
% title; the second line (mode number, frequency, IR/Raman intensity)
% is left untouched.
t = get(gca, 'Title');
t.String(1) = {'Violacein'};
set(gca, 'Title', t);

exportgraphics(gca, 'test_2_mode_91.pdf', 'ContentType', 'vector');

%% 3) Atomic charges -- Mulliken charges with a dipole moment overlay
G16_charges(filename, 'Plot', true, 'ShowDipole', true);
set(gca, 'View', [view_az, view_el]);

t = get(gca, 'Title');
t.String = {'Violacein - Mulliken Charges (atom)'};
set(gca, 'Title', t);

exportgraphics(gca, 'test_2_charges.pdf', 'ContentType', 'vector');

fprintf('\nDone. Figures: structure, mode 91, Mulliken charges.\n');
fprintf('Saved: test_2_mol.pdf, test_2_mode_91.pdf, test_2_charges.pdf
   \n');
```

16.2 example_2.m — Infrared and Raman spectra

Demonstrates `G16_spectra` two ways: a manual plot built directly from its raw `sp.x`/`sp.Raman_cont` output fields, and the function's own built-in two-panel Raman+IR auto-plot (`'plot', true`), restyled afterwards with larger, bold axis labels for a print-ready figure. Both are exported to PDF.

```
%% G16 Toolbox -- Example #2: Infrared and Raman spectra
%
%   Demonstrates G16_spectra on a real DFT calculation, two ways:
%     1) Manual plot -- build a single Raman continuum plot directly
%        from the raw sp.x/sp.Raman_cont fields returned by
   G16_spectra.
%     2) Built-in plot -- let G16_spectra draw its own two-panel
%        Raman+IR figure ('plot', true), then restyle the axes/labels
%        afterwards (larger, bold text) for a print-ready figure.
%
%   Data file: test_2.out -- DFT geometry optimisation of Violacein.
%   Reference: G. Cassone et al., Phys. Chem. Chem. Phys.,
%              doi: 10.1039/d6cp01164k
%
%   Requirements: G16/ on the MATLAB path, test_2.out in the current
%   folder. Running this script creates two PDFs in the current
%   folder: test_2_raman_manual.pdf, test_2_ir_raman.pdf.
```

```
clear; close all; clc
filename = 'test_2.out';
%% 1) Manual plot -- Raman continuum only, from the raw sp fields
sp = G16_spectra(filename);
fig1 = figure('Color', 'white');
plot(sp.x, sp.Raman_cont, 'LineWidth', 2);
xlabel('Wavenumber (cm^{-1})', 'FontWeight', 'bold');
ylabel('Raman activity (A^4 AMU^{-1})', 'FontWeight', 'bold');
set(gca, 'FontSize', 12);
exportgraphics(fig1, 'test_2_raman_manual.pdf', 'ContentType', 'vector
    ');
%% 2) Built-in two-panel plot (Raman + IR), then restyled
% 'FWHM' broadens the Lorentzian lines and 'xmin' crops the noisy
% low-frequency tail; 'plot', true triggers G16_spectra's own
% two-subplot Raman/IR figure (see G16_plot_spectra, local to
% G16_spectra.m).
G16_spectra(filename, 'FWHM', 15, 'xmin', 400, 'plot', true);
fig2 = gcf;
% G16_spectra creates the Raman axes first, then the IR axes below it;
% findobj returns them in reverse creation order, so flip to restore
% [Raman; IR].
axs      = flipud(findobj(fig2, 'Type', 'axes'));
ax_raman = axs(1);
ax_ir    = axs(2);
% Upsize the auto-generated labels (FontSize 10, regular weight) to a
% bolder, larger style for a print-ready figure.
set([ax_raman, ax_ir], 'FontSize', 12);
xlabel(ax_raman, 'Wavenumber (cm^{-1})',         'FontWeight', 'bold',
    'FontSize', 12);
ylabel(ax_raman, 'Raman activity (A^4 AMU^{-1})', 'FontWeight', 'bold
    ', 'FontSize', 12);
xlabel(ax_ir,    'Wavenumber (cm^{-1})',         'FontWeight', 'bold',
    'FontSize', 12);
ylabel(ax_ir,    'IR intensity (KM mol^{-1})',    'FontWeight', 'bold
    ', 'FontSize', 12);
exportgraphics(fig2, 'test_2_ir_raman.pdf', 'ContentType', 'vector');
fprintf('\nDone. Figures: manual Raman plot, built-in IR+Raman plot.\n
    ');
fprintf('Saved: test_2_raman_manual.pdf, test_2_ir_raman.pdf\n');
```

16.3 example_3.m — Energetics

Demonstrates `G16_orbital_energies`, `G16_energy`, and `G16_convergence`, renders the HOMO-LUMO diagram via `G16_draw_orbital`, and combines all three structs into a one-line summary — a small-scale taste of what `G16_read_all`/`G16_write_report` do at full scale across every extraction function.

```
%% G16 Toolbox -- Example #3: Energetics
%
%   Demonstrates G16_orbital_energies, G16_energy, and G16_convergence
%   on a real DFT calculation, then combines their results into a
%   one-line summary -- a small taste of what G16_read_all/
%   G16_write_report do at full scale across every extraction function
    .
%   Also renders the HOMO-LUMO diagram via G16_draw_orbital.
%
```

```
%   Data file: test_2.out -- DFT geometry optimisation of Violacein.
%   Reference: G. Cassone et al., Phys. Chem. Chem. Phys.,
%              doi: 10.1039/d6cp01164k
%
%   Requirements: G16/ on the MATLAB path, test_2.out in the current
%   folder. Each function already prints its own formatted summary to
%   the command window (shown below as a reference for what to expect
%   on this file); this script does not repeat those printouts.
   Running
%   it creates test_2_orbital_diagram.pdf in the current folder.

clear; close all; clc
filename = 'test_2.out';
%% 1) HOMO-LUMO gap -- detailed data in the oe structure
oe = G16_orbital_energies(filename);
% Console output for this file:
%   -- G16_orbital_energies (step 3/3): test_2.out --
%     HOMO = -0.190210 Ha  (-5.1759 eV)
%     LUMO = -0.096890 Ha  (-2.6365 eV)
%     Gap  =  0.093320 Ha  ( 2.5394 eV)

% Orbital energy-level diagram (title defaults to the source filename)
G16_draw_orbital(oe);
exportgraphics(gcf, 'test_2_orbital_diagram.pdf', 'ContentType', '
   vector');
%% 2) Energy and thermochemistry -- detailed data in the en structure
en = G16_energy(filename);
% Console output for this file:
%   -- G16_energy: test_2.out --
%     Method : RB3LYP
%     SCF    : -1160.20276011  Ha
%     ZPE    : +0.29182000  Ha   (766.17 kJ/mol)
%     E0+ZPE : -1159.91094000  Ha
%     H      : -1159.88988100  Ha
%     G      : -1159.96045200  Ha
%     S      :  621.4460  J/(mol.K)
%     T = 298.15 K,  P = 1.00000 atm

%% 3) Geometry-optimisation convergence -- detailed data in the cv
   structure
cv = G16_convergence(filename);
% Console output for this file:
%   G16_convergence: test_2.out
%     Steps read  : 11
%     Converged   : YES  (step 10)
%     Last step   : MaxF=1.00e-06 (thr 1.50e-05)  RMSF=0.00e+00 (thr
   1.00e-05)
%                   MaxD=1.24e-04 (thr 6.00e-05)  RMSD=2.50e-05 (thr
   4.00e-05)

%% 4) Combine all three into a one-line summary
if cv.converged
    conv_label = sprintf('YES (step %d/%d)', cv.conv_step, cv.Nsteps);
else
    conv_label = sprintf('NO (%d steps read)', cv.Nsteps);
end
fprintf('\nSummary for %s:\n', filename);
fprintf('  HOMO-LUMO gap : %.3f eV\n', oe.gap_eV);
```

```
if en.has_thermo
    fprintf('  Gibbs energy   : %.2f kJ/mol\n', en.G_kJ);
else
    fprintf('  Gibbs energy   : n/a (no frequency/thermochemistry job)\
       n');
end
fprintf('  Converged      : %s\n', conv_label);
```

16.4 example_4.m — Recovering and restyling graphics handles

Shows how to reach back into a figure already drawn by `G16_draw_molecule` and restyle every part of it after the fact (title, axis labels, legend, atom spheres, bond lines, atom-label text), rather than only being able to set style options at call time.

> **The key gotcha this example exists to document:** most of the graphics objects `G16_draw_molecule` creates (all bond lines, and every atom sphere after the first drawn for a given element) are created with `'HandleVisibility', 'off'`, so they do not clutter the legend or turn up in a plain `findobj` search. Reaching them requires `findall` instead of `findobj`. Title, axis labels, and the legend itself keep the default (`'on'`) visibility, so either function finds those. The script also re-enables the axes (`axis(ax, 'on')`) to reveal the X/Y/Z labels, which `G16_draw_molecule` always creates but hides by calling `axis(ax, 'off')` internally; and separates the atom-label text objects from the title/axis-label text objects (all four share `Type = 'text'`) with a `setdiff`.

```
%% Example 4
%% Recover graphics handles from G16_draw_molecule and restyle them
mol = G16_structure('test_2.out');
G16_draw_molecule(mol, 'Title', 'Violacein');

ax  = gca;
fig = gcf;

%% 1) Title -- always visible (HandleVisibility default 'on')
title(ax, 'Violacein (DFT-optimised)', 'FontSize', 13, ...
      'FontWeight', 'bold', 'Interpreter', 'tex');

%% 2) Axis labels -- G16_draw_molecule calls axis(ax,'off'), so labels
%    exist but are not shown until the axis is switched back on
axis(ax, 'on');
xlabel(ax, 'X (\AA)', 'FontSize', 11, 'Interpreter', 'latex');
ylabel(ax, 'Y (\AA)', 'FontSize', 11, 'Interpreter', 'latex');
zlabel(ax, 'Z (\AA)', 'FontSize', 11, 'Interpreter', 'latex');

%% 3) Legend -- created with default HandleVisibility, findobj works
   fine
hleg = findobj(fig, 'Type', 'Legend');
set(hleg, 'FontSize', 10, 'Location', 'northeast', 'Box', 'on');

%% 4) Atom spheres (Surface objects) -- use findall: most have
%    HandleVisibility 'off' (only the first atom per element is 'on')
hatoms = findall(ax, 'Type', 'Surface');
set(hatoms, 'FaceAlpha', 0.9, 'EdgeColor', 'none');

%% 5) Bond lines -- all created with HandleVisibility 'off', findall
   required
hbonds = findall(ax, 'Type', 'Line');
```

```
set(hbonds, 'LineWidth', 2.5, 'Color', [0.35 0.35 0.35]);

%% 6) Atom-label text objects only, excluding Title/XLabel/YLabel/
   ZLabel
%    (these are also Type='text', so must be explicitly subtracted out
   )
htitle_h = get(ax, 'Title');
hxlab    = get(ax, 'XLabel');
hylab    = get(ax, 'YLabel');
hzlab    = get(ax, 'ZLabel');
halltext = findall(ax, 'Type', 'Text');
hatomlbl = setdiff(halltext, [htitle_h; hxlab; hylab; hzlab]);
set(hatomlbl, 'FontSize', 8, 'FontWeight', 'bold');

%% 7) Background / figure colour
set(ax,  'Color', [1 1 1]);
set(fig, 'Color', [1 1 1]);

exportgraphics(fig, 'test_2_mol_restyled.pdf', 'ContentType', 'vector
   ');
```

17 References

Background reading for the physics/mathematics behind `MOSurface`'s real-space evaluation: Gaussian-type-orbital point evaluation (§9.1) and the analytic McMurchie-Davidson ESP integrals (§9.3).

1. S. F. Boys, "Electronic Wave Functions I. A General Method of Calculation for the Stationary States of Any Molecular System," *Proc. R. Soc. Lond. A* **200**, 542–554 (1950). — Introduces Gaussian-type orbitals for molecular electronic structure; origin of the GTO normalization constant used throughout `G_draw_mo_surface`.
2. T. Helgaker, P. Jørgensen, J. Olsen, *Molecular Electronic-Structure Theory*, Wiley (2000). — Standard graduate reference for Gaussian-type-orbital integrals, the Gaussian product theorem, and solid harmonics; useful background for §9.1 and (Ch. 9) the McMurchie-Davidson Hermite-Gaussian integral scheme used by `G_draw_esp_surface` (§9.3).
3. L. E. McMurchie, E. R. Davidson, "One- and Two-Electron Integrals over Cartesian Gaussian Functions," *J. Comp. Phys.* **26**, 218–231 (1978). https://doi.org/10.1016/0021-9991(78)90092-X — The original Hermite-Gaussian expansion/recursion scheme for analytic Gaussian integrals; the direct source for the exact electronic-ESP evaluation in `G_draw_esp_surface` (§9.3).
4. H. B. Schlegel, M. J. Frisch, "Transformation Between Cartesian and Pure Spherical Harmonic Gaussians," *Int. J. Quantum Chem.* **54**, 83–87 (1995). https://doi.org/10.1002/qua.560540202 — The original general formula for Cartesian ↔ pure spherical-harmonic Gaussian transformation coefficients, from one of Gaussian's own lead developers.
5. C. Ribaldone, J. K. Desmarais, "Spherical to Cartesian Coordinates Transformation for Solid Harmonics Revisited: Construction of the Hartree Potential," *arXiv:2412.16733* (2024). https://arxiv.org/abs/2412.16733 — Explicit, tabulated real-solid-harmonic polynomial coefficients up to $\ell = 10$; the direct source for the pure D/F/G formulas implemented in `G_draw_mo_surface`.
6. T. Verstraelen *et al.*, *IOData* (theochem/iodata), https://github.com/theochem/iodata, https://iodata.readthedocs.io. — Reference implementation and documentation of Gaussian's native `.fchk` basis-function ordering convention (including the Cartesian $L \geq 4$ shell reversal), used to verify the shell-component ordering in `G_draw_mo_surface`.

18 Publication and licensing

18.1 Declaration of Generative AI and AI-assisted technologies in the writing process

During the preparation of this work, the author(s) used Claude (Anthropic) to assist with code drafting, refactoring, and documentation of the G09/G16 toolbox and its associated user manual. After using this tool, the author(s) reviewed and edited the content as needed and take full responsibility for the content of the publication.

18.2 Note on the Use of AI-Assisted Development Tools

Portions of the codebase and documentation described in this manual — including the `G09_`/`G16_` MATLAB function pairs, the `g16toolbox` Python package, the `G09_modeViewer.m`/ `G16_modeViewer.m` interactive GUIs, and this reference manual — were developed with the assistance of Claude (Anthropic), an AI-based coding tool, used interactively under the direct supervision of the author. All AI-generated code was tested against reference Gaussian 09/16 output files and reviewed by the author prior to inclusion. Algorithm design, scientific validation, and correctness of results remain the sole responsibility of the author.

18.3 Scope and disclaimer

This toolbox is provided as a research and productivity aid for parsing, analysing, and visualising Gaussian 09/16 output files — it is not a validated or certified computational-chemistry package, and its numerical output has not been benchmarked beyond the specific test cases described in this manual and its companion `G_Utility` manual (which carries its own, matching "Scope and disclaimer" note). A number of functions rely on geometric or heuristic approximations rather than a direct read of a corresponding Gaussian-computed quantity: for instance, bond order in `G09_draw_molecule`/`G16_draw_molecule` (§7.1) is estimated purely from bond length, not from an actual Gaussian Wiberg/NBO bond-order analysis, and C–N bonds are deliberately never classified as double even when geometrically short, since a single length threshold cannot separate an aromatic C–N from a genuine C=N. Every such approximation is documented, together with its known limitations, at the point in this manual (or the `G_Utility` manual) where the relevant function is described. Users remain responsible for independently verifying any result before relying on it for a scientific conclusion or publication — the same standard that would be applied to the output of any third-party analysis tool. This note is a scientific/methodological caveat and does not replace the legal warranty disclaimer already given in the `LICENSE` file distributed with the source code.

18.4 Licence

This software is released under the **BSD 3-Clause License**, an OSI-approved open source license accepted by SoftwareX. See the `LICENSE.txt` file distributed with the source code for the full licence text.

18.5 Citation

If this toolbox contributes to published research, please acknowledge it by citing the source repository:

> S. Trusso, *G09/G16 Toolbox: MATLAB and Python libraries for parsing, analysing and visualising Gaussian 09/16 output files*, https://github.com/nello62/Gaussian09G16_output_Matlab_parser (accessed November 12, 2026).